****DISCLAIMER: POTREBBERO ESSERCI ERRORI O IMPRECISIONI, FEEL FREE TO CORREGGERE****

26/7/2022

Consider the following 4 activation functions: Sigmoid, Tanh, Linear, ReLU. Discuss for each of them when you should use it and why

These activation functions are all commonly used in Neural networks to introduce non linearities in the computation. The choice is done according to the specific task the network will be designed to perform. As an example, sigmoid and tanh, which are functions that map the values in an interval respectively between 0 an 1 and 1 and -1, are most suitable for classification tasks, while ReLU is used commonly in regression ones. Linear function may be used for regression but is not very suitable because it does not serve the purpose activations are made to achieve.
Also, sigmoid and tanh are possibly related to the vanishing gradient issues, since the derivatives are often very small values that are backpropagated in the deeper layers, while ReLU solves this problem by having its derivatives in 1 and 0 but may lead to dying neurons because of that. This is why it's more used in RNNs.

Why is it so important to choose a proper initialization, i.e., what could it happen if it is wrong? Which methods do you suggest for network initialization?

Weight initialization is important because it directly influences the way a network can learn. An appropriate weight initialization will lead to efficient training, while an inefficient one may cause overfitting or underfitting. Some choices for weight initialization can be:

- All 0s, not good because all the weights learn in the same way and thus they're incapable of extracting any feature;
- High values, not good because big values may lead to overfitting or cause vanishing gradients because they're closer to saturation zones in the derivatives of some functions;
- Small values according to a gaussian distribution centered in 0 and with unitary variance, good for small nets but may lead to vanishing gradient in deeper ones;
- Techniques such as Xavier and Glorot, which are good because they try to standardize the weight initialization in a way that their variance is kept under control, so that they'll not grow too much but still be able to learn effectively, by using a distribution with certain characteristics (variance proportional to the number of inputs) from which selecting the values;
- Transfer learning or fine tuning, since the nets that are transferred are assumed to be reliable;
- Autoencoders, taking the encoder after training, can reduce overfitting;

Mark the TRUE statements among the following

a. The exploding gradient is not due to the classical sigmoidal/tanh/ReLU/linear activation functions

b. Classical recurrent neural networks (RNN), even if properly initialized, can learn only short sequences because of the exploding gradient

c. Sigmoidal activation functions are the cause of the vanishing gradient

d. Long Short-Term Memories (LSTM) prevent both exploding and vanishing gradient issues thanks to the Constant Error Carousel.

e. ReLU activation function can speed up training

f. Skip connections can increase the vanishing gradient issue

Check all the following statements which are true. Scegli una o più alternative:

a. CNNs can only take as input images

b. CNNs are typically used to solve visual recognition problems taking as input an image, while MLP are very poor in this

c. There are no ways to feed an image to a MLP

d. CNN are always deeper than MLP

e. A CNN can be seen as a special MLP network, provided some constraints on weights

Please check all the following networks that can be entirely trained in an unsupervised manner. Assume a suitable training set is provided. Scegli una o più alternative:

a. Yolo

b. An instance segmentation network

c. A Deep Neural Network that retrieves from a database of face, the most similar face to a query one

d. A network reading the time given the picture of an analogic clock

e. An autoencoder

f. The U-net segmenting specific cells in biomedical images.

g. A network providing an heatmap where humans are in the image

h. A GAN to generate faces

Commentato [BI1]: Questa perché puoi farlo con un autoencoder che estrae la latent feature representation (io avrei detto con la distanza ma vb)

Parametric Search of CNN with multiple Conv2D layers, MP, Dense and Concat

```
# Build the neural network layer by layer
input_layer = tfkl.Input(shape=input_shape,name='input')

conv_in = tfkl.Conv2D(filters=16,kernel_size=1,strides=1,padding='same',name='conv_in')(input_layer)
conv_1 = tfkl.Conv2D(64,1,1,padding='same',activation='relu',name='conv_1')(conv_in)
conv_2 = tfkl.Conv2D(128,3,4,padding='same',activation='relu',name='conv_2')(conv_1)
conv_3 = tfkl.Conv2D(32,1,1,padding='same',activation='relu',name='conv_3')(conv_in)
conv_4 = tfkl.Conv2D(64,3,2,padding='same',activation='relu',name='conv_4')(conv_3)
conv_5 = tfkl.Conv2D(128,3,2,padding='same',activation='relu',name='conv_5')(conv_4)
mp = tfkl.MaxPooling2D((4,4), name='mp')(conv_in)
conv_7 = tfkl.Conv2D(128,1,1,padding='same',activation='relu',name='conv7')(mp)
concat = tfkl.Concatenate(axis=-1,name='concat')([conv_2,conv_5,conv_7])

gap = tfkl.GlobalAveragePooling2D(name='gap')(concat)
dense = tfkl.Dense(512,activation='relu', name='dense')(gap)
dropout = tfkl.Dropout(.5,name='dropout')(dense)
output = tfkl.Dense(100, activation='softmax', name='output')(dropout)

# Connect input and output through the Model class
model = tfk.Model(inputs=input_layer, outputs=output, name='model')

# Compile the model
model.compile(loss=tfk.losses.CategoricalCrossentropy(), optimizer=tfk.optimizers.Adam(), metrics='accuracy')

# Return the model
return model

input_shape = (64,64,3)
model = build_model(input_shape)
model.summary()
```

Remember that concat stacks together the feature maps

Layer (type)	Output Shape	Param #	Connected to
input (InputLayer)	(None, 64, 64, 3)	0	[]
conv_in (Conv2D)	(None, 64, 64, 16)	64	['input']
conv_3 (Conv2D)	(None, 64, 64, 32)	544	['conv_in']
conv_1 (Conv2D)	(None, 64, 64, 64)	1088	['conv_in']
conv_4 (Conv2D)	(None, 32, 32, 64)	18496	['conv_3']
mp (MaxPooling2D)	(None, 16, 16, 16)	0	['conv_in']
conv_2 (Conv2D)	(None, 16, 16, 128)	73856	['conv_1']
conv_5 (Conv2D)	(None, 16, 16, 128)	73856	['conv_4']
conv7 (Conv2D)	(None, 16, 16, 128)	2176	['mp']
concat (Concatenate)	(None, 16, 16, 384)	0	['conv_2', 'conv_5', 'conv7']
gap (GlobalAveragePooling2D)	(None, 384)	0	['concat']
dense (Dense)	(None, 512)	197120	['gap']
dropout (Dropout)	(None, 512)	0	['dense']
output (Dense)	(None, 100)	51300	['dropout']

Total params: 418,500
Trainable params: 418,500
Non-trainable params: 0

Search of receptive field

Standard formula of k+(x-1)*s for Conv layers

Consider the following sequence of layers

```
input_layer = tfkl.Input(shape=(64,64,1),name='input')
conv_1 = tfkl.Conv2D(filters=16,kernel_size=5,strides=1,padding='same',activation='relu',name='conv_1')(input_layer)
conv_2 = tfkl.Conv2D(128,3,1,padding='same',activation='relu',name='conv_2')(conv_1)
conv_3 = tfkl.Conv2D(32,1,1,padding='same',activation='relu',name='conv_3')(conv_2)
mp = tfkl.MaxPooling2D((2,2), name='mp')(conv_3)
conv_4 = tfkl.Conv2D(64,3,1,padding='same',activation='relu',name='conv_4')(mp)
mp = tfkl.MaxPooling2D((2,2), name='mp')(conv_4)
```

What is the receptive field at the central pixel of the activations after **the last mp**?

Scegli una o più alternative:

- [ ] a. 8 x 8
- [ ] b. 15x15
- [ ] c. 5 x 5
- [ ] d. 14 x 14
- [ ] e. Since there are two maxpooling, it is one quarter of the input size
- [x] f. 22 x 22 ✖
- [ ] g. The entire image
- [ ] h. 10x10

Your answer is incorrect.
La risposta corretta è:
14 x 14

What is word embedding required for? How does the model to implement it work?

Word embedding is a technique used to work with language processing neural networks. Before word embedding, methods like 1-hot encoding were used to turn the words of a vocabulary into numerical

values, but in this way dimensionality was very high and semantic closeness between words was not captured. Word embedding solves these issues by turning words into a latent representation that is not only lower dimensional than the original vocabulary encoding, but is also capable of capturing the meanings of similar words. It's usually performed through the use of neural networks that are capable of encoding. One famous model that works with embeddings is Neural Net Language Model, which is a model that's able to predict the words in sentences thanks to its ability of exploiting the connections found between them during training. This model uses different layers: an input layer for accepting the initial encoding of the word, an projection layer that maps the embedding to the vocabulary words, hidden layers to perform the computation, and an output layers that gives a probability distribution of words to see which ones are more suitable for the single example. It takes as input couples <word, context> in which the context is a sequence of other words, and makes predictions. Another similar model is word2vec, which removes the hidden layers and shares the projection layers among all examples, the output is still a probability of a word given the context, but is found considering both previous and successive context words and the values updated are the ones in the context vector.

Let's consider attention in the seq2seq model. To what extent, by explaining its functioning, you can say that the input is used as a context memory?

Seq2seq models are used to process sequential data like sentences. They're composed by an encoder-decoder structure and try to achieve a latent representation of the input in a way that they'll be able to use it efficiently in the decoding section. They do this by basing the outputs of the successive layers on the ones of the previous ones, to better capture the dependencies between them. Attention is used because it can be able to make the net perform better avoiding bottlenecks by giving different relevance to the sequence's sections. At each step, some attention weights are estimated considering the hidden state of the decoder and the hidden states of the encoder, and used to assign weighted importance to the input values. The whole input is then evaluated by the attention mechanism at each step of the sequence generation and because of this is used as a memory differently from the classic seq2seq model where it is canceled once the encoding is completed.

24/6/2022

What is the learning rate and how would you choose its value?

Learning rate is a pivotal part of a neural network. It's a hyperparameter that is used during the gradient descent to determine the entity of the step to take at every iteration of the weight update. It is used to multiply the value of the gradient with respect to the weight and thus modify the old one. It's important to choose it in a way that the net can learn efficiently, often using hyperparameter tuning. High values are not very suited, because they cause oscillations in convergence. Small values are a better choice but lead to slow convergence, and usually a fixed learning rate is discarded in favor of an adapting one. This adaptation can be performed with techniques such as Momentum, Adagard and Adam , which update the learning rate value considering the history of the gradient directions to be more resistant to direction variations (velocity) and also try to achieve a faster convergence.

Which of the following is true?

1. A low learning rate does not compromise training
2. A high learning rate might compromise training

3. The higher the learning rate the better the training
4. The learning rate should be reduced while training

Does learning rate have some relationship with the dying neurons issue? Why?

A possible cause of dying neurons is the activation function used in neurons, like ReLU. During gradient descent, the derivatives of the activations are considered, and this function can have them be 0 or 1. If the inputs are in saturation zones, the derivative will be equal to 0 for each neuron, and thus the weights will not be updated. In this case, no other operation can change the outcome, and the neurons will not be activated anymore. A too high learning rate may cause the values of the activations to assume saturation values and switch it off, so smaller rates are preferred. To prevent this, variations of the ReLU were introduced, such as Leaky ReLU, which adds steepness to the function in order to have the derivatives assume different values.

Parametric Search with Conv and Flatten and Add layers

```
import tensorflow as tf
tfk = tf.keras
tfkl = tf.keras.layers

# Exam 2022/06/27
classes = 10
filters = 16

# Feature Extractor
input_layer = tfkl.Input(shape=(128,256,1), name='input')
conv1 = tfkl.Conv2D(filters, kernel_size=3, strides=(1, 1),
padding='same', activation='relu', name='conv1')(input_layer)
mp1 = tfkl.MaxPooling2D(name='mp1')(conv1)
conv2 = tfkl.Conv2D(filters*2, 3, (1, 1), padding='same', activation='relu',
name='conv2')(mp1)
conv3 = tfkl.Conv2D(filters*2, 3, (1, 1), padding='same', activation='relu',
name='conv3')(conv2)
mp2 = tfkl.MaxPooling2D(name='mp2',pool_size=(4,4))(conv3)
conv4 = tfkl.Conv2D(filters*2, 1, (1, 1), padding='same', activation='relu',
name='conv4')(mp2)
conv5 = tfkl.Conv2D(filters*2, 1, (1, 1), padding='same', activation='relu',
name='conv5')(conv4)
add2 = tfkl.Add()([mp2, conv5])
rel = tfkl.ReLU()(add2)
conv6 = tfkl.Conv2D(filters, 3, (1, 1), padding='same', activation='relu',
name='conv6')(rel)
mp3 = tfkl.MaxPooling2D(name='mp3',pool_size=(4,4))(conv6)
flt = tfkl.Flatten(name='flatten')(mp3)
classifier = tfkl.Dense(filters, activation='relu', name='classifier')(flt)
output_layer = tfkl.Dense(classes, activation='softmax', name='output')
(classifier)

# Connect input and output through the Model class
residual = tfk.Model(inputs=input_layer, outputs=output_layer,
name='model170122')
```

Input(InputLayer)	(None, [128 256 1])	[0]
Conv1 (Conv2D)	(None, [128 256 16])	(3x3x1)x16
MaxPooling	(None, [64 128 16])	0
Conv2 (Conv2D)	(None, [64 128 32])	(3x3x16)x32 + 32
Conv3(Conv2D)	(None, [64 128 32])	(3x3x32)x32 + 32
MaxPooling2	(None, [16 32 32])	0
Conv4(Conv2D)	(None, [16 32 32])	(1x1x32x32)+32
Conv5(Conv2D)	(None, [16 32 32])	(1x1x32x32)+32
Add(MP2, Conv5)	(None, [16 32 32])	0
Relu	(None, [16 32 32])	0
Conv6(Conv2D)	(None, [16 32 16])	(3x3x32x16)+16
MaxPooling3	(None, [4 8 16])	0
Flatten	(None, 512)	0

Classifier	(None, 16)	16x512+16
Output	(None, 10)	10x16+10

Which of the following statements about Global Averaging Pooling is true?

1. GAP has no trainable parameters
2. GAP should not be used in networks containing a batch normalization layer
3. GAP is used in object detection networks
4. Networks provided with GAP need to be trained to return class activation mapping
5. Including a GAP in a CNN is a good way to make the CNN invariant to input size

Which of the following statements about instance segmentation and semantic segmentation is true?

1. Instance segmentation and semantic segmentation are different problems solved by the same networks
2. In order to train an instance segmentation network, it is necessary to fine tune a semantic segmentation network first
3. Instance segmentation network returns both segments and bounding boxes
4. Architectures for semantic segmentation can have a shape similar to a convolutional autoencoder
5. Fully convolutionalization is a way to modify without training a CNN classifier, to become a (very coarse) instance segmentation network

What is the vanishing gradient issue and what is its cause?

Vanishing gradient is an issue of NN, especially deep ones, in which the gradient becomes very small layer after layer when performing backpropagation. A possible cause may be a wrong weight initialization, but it is usually caused by the activation function choices: Sigmoid and Tanh, as an example, have very small derivative values in some intervals, and because they're multiplied by every previous layer's gradient, the initial layers can have very small gradients, leading to inefficient training. Functions like ReLU and mechanisms like the LSTM can solve this problem.

How do Long Short-Term Memories solve the vanishing gradient issue?

LSTM are a structure of RNN that are optimized to work with memory. They use cells that have a set of gates to keep a long term memory and a short term memory that will influence the output of the cell together with the input. They can solve the vanishing gradient because they use the Constant Error Carousel, which stabilizes the gradient to make the error always equal to 1 and uses linear activations. This is because their structure, as all kinds of deep recurrent nets, is prone to vanishing gradients, and this way its value is kept under control. The net is still able to learn because of the cell structure.

How do (sparse) Neural Autoencoders work? What is their use?

Autoencoders are a special kind of NN used to learn a representation of the input through which they'll be able to reconstruct it. It's called latent representation because it's generated by a feature extraction

network that keeps the most relevant information to generate a smaller portray of the data, and this information will be used by the decoder. They're used in tasks like word embedding and language processing, to reduce the dimensionality of the vocabulary through mapping or to capture the semantics of a sentence to be able to better predict the words in it. Another use is anomaly detection, noise reduction or weight initialization.

Describe the Word2Vec Continuous Bag of Words model

Word2Vec is a language processing model made by Google. It works using one hot encodings to generate word embeddings that better capture the semantic closeness between words. Its structure contains a projection layer which is used to perform a mapping between the embeddings and the 1-hots to try to predict the next word in the sentence via a softmax function, and thus the net is trained by using multiclass crossentropy. It learns both the embedding and uses them to make the most suited words according to the context by updating this vector considering the network's output. It can also implement attention mechanisms to be more efficient.

"The Word2Vect Continuous Bag of Words model is used to perform word embedding from the one-hot encoding to a dense vector representation. This embedding is obtained via unsupervised learning by predicting the one-hot encoding of a word from the one-hot encodings of the surrounding words. In particular each input one-hot encoded word is projected into a continuous vector and the n vectors obtained this way are averaged. This average is then used to predict the output word via softmax. The training is performed using multi class cross-entropy."

Why Word2Vec represents and efficient and effective way to encode words?

Word2Vec is more efficient than classic 1-hot encoding because the 1-hot has a dimensionality problem: a vocabulary usually contains a lot of words, and vectors will be very large and sparse. Predicting the probability for each word will require a lot of data. Also, 1-hot does not keep track of the semantics of words, because some of them may have a stronger bond than others. Working with word embeddings not only reduces the entity of the representation, because they're usually smaller than the originals, but also, because of context information, can consider the meanings of words to generate a latent space in which similar words are closer.

09/02/2022

What is Vanishing Gradient in RNN? How is it fixed in LSTM? Why does it fix the problem?

Vanishing gradient is a phenomenon that can occur in NN due to a number of causes. It's mostly due to the choice of activation functions. Sigmoid and Tanh, as an example, have their derivatives take values between 0 and 1. This becomes an issue in RNN since they're generally deeper than other structures and have the gradient of earlier timesteps computed using the activation values of the successive layers, which use the same values and thus make the gradient multiplied by a lot of small numbers. To prevent this, we can use ReLU activation, but LSTM solve the problem by acting on the architecture and using a cell structure that includes different gates. These gates are used to perform the computations and delete and update the long term memory and short term memory. They have a CEC that keeps the error fixed to 1 during the

computation of the gradient, so that it does not vanish, and use linear activations to prevent the issues sigmoid and tanh caused.

Parametric Search of CNN with Dense, CLayers and BatchNorm

Batch Normalization has 4 parameters for each slice in the depth volume of the map!

Dimension for transpose convolution found with the receptive field formula.

```
import tensorflow as tf
tfk = tf.keras
tfkl = tf.keras.layers

input_shape = (128,256,3)

input_layer = tfkl.Input(shape=input_shape, name='input')

conv1 = tfkl.Conv2D(filters = 16, kernel_size = (5,5), strides=(1,1), padding='same', acti
vation='relu', name='conv1')(input_layer)
mp1 = tfkl.MaxPooling2D(name='mp1')(conv1)

conv2 = tfkl.Conv2D(filters = 32, kernel_size = (3,3), strides=(1,1), padding='same', acti
vation='relu', name='conv2')(mp1)
mp2 = tfkl.MaxPooling2D(name='mp2')(conv2)
dropout = tfkl.Dropout(0.3)(mp2)

conv3 = tfkl.Conv2D(filters = 128, kernel_size = (1,1), strides=(1,1), padding='same', ac
tivation='relu', name='conv3')(dropout)
batchNorm = tfkl.BatchNormalization(name='batchNorm')(conv3) #this normalizes eac
h slice of the volume independently

convt1 = tfkl.Conv2DTranspose(filters = 32, kernel_size = (3,3), strides=(2,2), padding
='same', activation='relu', name='convt1')(batchNorm)
convt2 = tfkl.Conv2DTranspose(filters = 16, kernel_size = (3,3), strides=(2,2), padding
='same', activation='relu', name='convt2')(convt1)
output_layer = tfkl.Conv2DTranspose(filters = 3, kernel_size =(1,1), strides=(1,1), paddin
g='same', activation='sigmoid', name='output')(convt2)

model = tfk.Model(inputs=input_layer, outputs=output_layer, name='model')
# Consider now the execution of the following command
model.summary()
```

Solution (written by professor in a mail):

```
Layer (type) Output Shape Param #
=================================================================
input (InputLayer) (None, 128, 256, 3) 0
conv1 (Conv2D) (None, 128, 256, 16) 16*5*5*3+16=1216
mp1 (MaxPooling2D) (None, 64, 128, 16) 0
conv2 (Conv2D) (None, 64, 128, 32) 32*3*3*16+32=4640
mp2 (MaxPooling2D) (None, 32, 64, 32) 0
dropout_8 (Dropout) (None, 32, 64, 32) 0
conv3 (Conv2D) (None, 32, 64, 128) 128*1*1*32+128=4224
batchNorm (None, 32, 64, 128) 128*4=512
convt1 (Conv2DTranspose) (None, 64, 128, 32) 32*3*3*128+32=36896
convt2 (Conv2DTranspose) (None, 128, 256, 16) 16*3*3*32+16=4624
output (Conv2DTranspose) (None, 128, 256, 3) 3*1*1*16+3=51
=================================================================
Total params: 52,163
Trainable params: 51,907
```

Receptive Field

What is the receptive field of a pixel at the centre of the output of the batchnorm layer?

Solution: 12x12

Start from the end (in this case BatchNorm layer) (init) -> 1
Conv2D(1x1) 1 +(1-1) -> 1
MaxPool 1*2 -> 2
Conv2D(3x3): 2 + 3 -1 -> 4
MaxPool: 4*2 -> 8
Conv2D(5x5) -> 12

In fact, the network outputs a tensor having the same size as the original image and it's trained to minimise crossentropy. Therefore, the network is a segmentation network that any neural network expert like you are expected to be, would train that to solve a three-class image segmentation problem.

- ☐ determining how many persons appears in the image
  Wrong: that's a regression problem on the image
- ☑ determining the image regions covered by cars, by persons and anything else
  that's correct
- ☐ determining whether it is winter or summer, whether it is raining or not
  Wrong: that's a three-class classification problem
- ☐ determining which pixels contain a human, a car and what is the temperature in there
  Wrong: here the output seems compatible with the network architecture but the temperature is not a categorical variable, so that's not the right network for this task
- ☐ determining where cars are parked in the image
  Wrong: this can be either interpreted as a two-class classification problem or a localization problem, which are not segmentation problems
- ☐ determining where there are empty parking slots
  Wrong: this can be either interpreted as a two-class classification problem or a localization problem, which are not segmentation problems
- ☐ tracking persons, cars and buses moving in the scene
  Wrong: this requires drawing bounding boxes or in any case identifying instances, which is not what image segmentation does
- ☐ determining the locations of: each person, each dog, each car in the scene.
  Wrong: this requires drawing bounding boxes or in any case identifying instances, which is not what image segmentation does
- ☑ determining all the pixels covered by road, sky or others
  that's correct.

Consider the Inception Net module. Which of the following statements are true?

(1 punto)

- ☑ it uses multiple convolutional filters of different sizes in parallel
- ☐ it was the first to introduce skip connections
- ☐ it has been the first module used for semantic segmentation
- ☐ it was the first learnable upsampling filter
- ☑ it leverages 1x1 convolutions to reduce the computational burden

What are GANs? Briefly introduce the training process

GANs are neural networks that belong to the class of generative models. They are trained to generate new samples of a dataset on which training was performed. They do this exploiting an architecture with 2 components:

- Generator, which, given some random noise as input, is able to create new samples of the interested set;
- Discriminator, which checks whether or not the generator's output is correct;

Training is performed un an adversarial manner: the generator and the discriminator are both initially trained in a traditional way to learn the dataset's features and be able to get a correct representation of it. After this they're trained together: the generator, which can be seen as a kind of decoder, generates from the noise a sample thanks to the learned latent portrait, and then this image is fed to the discriminator, which is a classificatory that determines if it can pass. The loss function takes into account both of the outcomes: the discriminator tries to maximize the difference between classification of correct and incorrect images, while the generator wants to minimize it.

What are Neural Turing Machines? How does attention work in this kind of models?

Neural Turing Machines are models of recurrent networks that are able to access an external memory bank, They do this with operations of read and write, which are performed using computation over the current state of the net and the past states. The read and write heads are implemented using a set of weights that act as a soft attention mechanism. The addressing parameters are learned by the NN using the state of the network, that generates a distribution, used to access differently the sections of the memory matrix. The weights indicate how much to write in the memory cells. This way, we can read and write everywhere but to a different extent. Attention can be content based or location based.

17/01/2022

Which of these techniques might help with overfitting?
(1 punto)

- [x] Weight Decay
- [x] Early Stopping
- [x] Dropout
- [ ] ReLu and Leaky ReLu
- [ ] Stochastic Gradient Descend
- [x] Xavier Initialization
- [x] Batch Normalization

Which of these techniques might help with vanishing gradient?
(1 punto)

- [ ] Dropout
- [x] ReLu and Leaky ReLu
- [ ] Early Stopping
- [x] Xavier Initialization
- [ ] Stochastic Gradient Descend
- [ ] Weight Decay
- [x] Batch Normalization

What is the dying neuron problem and how would you fix it?

The dying neuron problem happens when the gradient considering a set of weights goes to 0 and it's no longer possible to update them, thus becoming dead. This can happen because activation functions like ReLU are used: this has its derivative assume values of 1 and 0 only, and if the values are in saturation zones the output can often go to 0. This value is multiplied to all others due to backpropagation, and nullifies the computation. A possible solution can be adopting variants of the ReLU such as Leaky or using a LSTM structure.

Parametric Search of NN

```
tfk = tf.keras
tfkl = tf.keras.layers

input_shape = (256, 256, 3);

# Build the neural network layer by layer
input_layer = tfkl.Input(shape=input_shape, name='Input')

conv1 = tfkl.Conv2D(filters=8, kernel_size=(5, 5), strides = (1, 1),padding = 'same',activ
ation = 'relu', name='conv1')(input_layer)
pool1 = tfkl.MaxPooling2D(pool_size = (2, 2), name='mp1')(conv1)

conv2 = tfkl.Conv2D(filters=32,kernel_size=(5, 5),strides = (2, 2),padding = 'same',activ
ation = 'relu', name='conv2')(pool1)
pool2 = tfkl.MaxPooling2D(pool_size = (2, 2), name='mp2')(conv2)

btchNorm = tfkl.BatchNormalization(name='batchNorm')(pool2) #this normalizes each
slice of the volume independently

conv3 = tfkl.Conv2D(filters=64,kernel_size=(3, 3),strides = (2, 2),padding = 'same',activ
ation = 'relu', name='conv3')(btchNorm)
pool3 = tfkl.MaxPooling2D(pool_size = (2, 2), name='mp3')(conv3)

flattening_layer = tfkl.Flatten(name='Flatten')(pool3)

dropout1 = tfkl.Dropout(0.3)(flattening_layer)
dense1 = tfkl.Dense(units=64, name='Dense1', activation='relu')(dropout1)

dropout2 = tfkl.Dropout(0.3)(dense1)
dense2 = tfkl.Dense(units=32, name='Dense2', activation='relu')(dropout2)

output_layer = tfkl.Dense(units=2, activation='linear', name='Output')(dense2)

# Connect input and output
model = tfk.Model(inputs=input_layer, outputs=output_layer, name='model')

# Consider now the execution of the following command
model.summary()
```

Layer (type)	Output Shape	Param #
Input (InputLayer)	(None, [256 256 3])	[0]
conv1 (Conv2D)	(None, [256 256 8])	[8(5x5x3+1)]
mp1 (MaxPooling2D)	(None, [128 128 8])	[0]
conv2 (Conv2D)	(None, [64 64 32])	[32(5x5x8+1)]
mp2 (MaxPooling2D)	(None, [32 32 32])	[0]
batchNorm	(None, [32 32 32])	[32*4]
conv3 (Conv2D)	(None, [16 16 64])	[64(3x3x32+1)]
mp3 (MaxPooling2D)	(None, [8 8 64])	[0]
Flatten (Flatten)	(None, [8*8*64])	[0]
dropout (Dropout)	(None, [8*8*64])	[0]
Dense1 (Dense)	(None, [64])	[(8*8*64+1)*64]
dropout (Dropout)	(None, [64])	[0]
Dense2 (Dense)	(None, [32])	[(64+1)*32]
Output (Dense)	(None, [2])	[(32+1)*2]

What can the NN be used for?

Spiegazione: rete usata per regressione su max 2 valori, è possibile anche determinarne il colore perché assimilabile ad un valore numerico in un largo intervallo.

- [ ] inference on whether the car is red or not
- [x] inference on height of the driver and his income
- [ ] inference on academic degree of the car owner
- [x] inference on the cost
- [ ] inference on the cost, the horse power and the maximum speed
- [x] inference on the maximum speed and the colour
- [ ] inference on the brand and the model
- [x] inference on its maximum speed

Augmentation to perform on image of clock

Assume you want to develop a system to automatically read the hours from your old w all clock as in the picture.

Now, assume you place a webcam in front of your wall clock, gather a lot of images wit h their acquisition time and you are ready to start training your CNN.

You don't have many images however, and you want your network to be robust to diff erent positioning of the webcam in front of the wall clock, different weather / light con ditions...

What kind of data augmentation **should be avoided** during training?

(1.5 punti)

- [ ] translation
- [x] scaling of y axis
- [x] rotation
- [x] vertical flip
- [ ] noise addition
- [x] scaling of x axis
- [x] horizontal flip
- [ ] change in brightness
- [ ] image scaling

Briefly describe what Siamese Networks are and what they are used for.

Siamese networks are a kind of network that is used to perform classification tasks. They use a distance measure to predict the label of an example. They are composed of 2 identical NN which are fed different samples from a dataset. Their structure is the same, they extract a latent representation and then compare it using a difference (distance in the latent space). Their loss function has the objective of maximizing the distance between different images and minimizing the one between samples from the same class.

What is the goal of Word Embedding and what motivates it?

Word Embedding is a technique used when working with language processing networks. Words have to be given a numerical representation to be able to be fed as input, and choosing how to do it is not easy: initially, 1-hot encoding was used, but this representation suffers from 2 main issues:

- Dimensionality: vocabularies are very large and vectors are sparse. An effective use of the dataset will require a lot of information to be able to make optimal predictions;
- Semantics, this kind of representation does not capture the meaningful information that word semantics can have. Two similar words are completely unrelated.

Word Embeddings solve this problem by using a latent representation of words that is lower in dimensionality and captures the meanings, mapping the words in a latent space in which closer words are "physically" closer. They do this using a NN structure such as an encoder, and using a projection of the embedding over the vocabulary to learn important information from the context.

Describe the word2vec network architecture.

This architecture is used to process sequences of text. It can be used to work with word embeddings and make prediction over the next word in a sentence. The computation is based on <target, context> pairs in which the net tries to predict the next word in a sequence considering the context vector, using both previous and successive elements. The net uses word embeddings to achieve a more meaningful representation of the input data: these embeddings are learned and projected in a layer that does the mapping between them and the original vocabulary. This outcome is used by a classification section to find a probability distribution over the words in the vector given the context. According to this value, the context vector is adjusted.

Can word embedding overfit? If so, how is it possible?

It may be possible for word embeddings to overfit if the dataset is not large enough or if the NN is too complex. This way, the embeddings are learned in a way that is not able to generalize and make predictions over unseen data, since the latent space representation is too specific. Different techniques to prevent overfitting can be used.

How are sentences embedded in seq2seq models?

This particular NN uses an encoder-decoder structure to learn a latent representation of the input words. It is used to work with sequences of words, learning an effective and lower dimensional portrait of the input to generate different output sequences. This is done by considering the outputs of the previous layers when generating new ones. The two sections are first trained over the dataset, so that they're able to learn the relationships in it, and at inference the outcome is generated starting from the learned mapping. Embeddings use dense vectors to represent words and some special characters like PAD, EOS, UNK, SOS/GO.

How does the attention mechanism work?

Seq2Seq modelling designed to generate an output sequence from an input sequence. It is a classical encoder-decoder architecture. The encoder takes as input a sequence and maps it to a fixed-length context vector, which is passed to the decoder. The decoder takes this vector and generates an output sequence.

Attention in seq2seq models allows networks to give more importance to certain input segments than others. It is implemented by considering the current hidden state of the decoder and the hidden states of the encoder and concatenate them. These values are then used to generate a distribution that's used to weight the sections of the input, so the network can learn which ones are more relevant for the output predictions.

08/07/2021 mio comple 😊 ma tde difficile ☹

What is word embedding and what it is used for?

Word Embedding is a technique used when working with language processing networks. Words have to be given a numerical representation to be able to be fed as input, and choosing how to do it is not easy: initially, 1-hot encoding was used, but this representation suffers from 2 main issues:

- Dimensionality: vocabularies are very large and vectors are sparse. An effective use of the dataset will require a lot of information to be able to make optimal predictions;
- Semantics, this kind of representation does not capture the meaningful information that word semantics can have. Two similar words are completely unrelated.

Word Embeddings solve this problem by using a latent representation of words that is lower in dimensionality and captures the meanings, mapping the words in a latent space in which closer words are "physically" closer. They do this using a NN structure such as an encoder, and using a projection of the embedding over the vocabulary to learn important information from the context.

Is word embedding a supervised or an unsupervised machine learning task? Why?

Word embedding is considered to be an unsupervised learning task. This is because it tries to learn a latent representation of the input words given the context, also using an encoding structure, without any labelling from the outside. This embedding is done in the input to hidden layer section, the projection, and this part is done without external aids. This is why even if models like word2vec are considered supervised, embedding is not.

How is the word embedding loss function defined?

The loss function in word embedding is defined as, if we consider as an example the CBOW model, the probability of a word given the context. This is the negative log likelihood of the target over the context vector, and the goal is minimizing it through gradient descent approaches. The context vector is adjusted according to the value of the loss function.

Can word embedding overfit? Motivate your answer and, in case it does, suggest a way to detect it.

It may be possible for word embeddings to overfit if the dataset is not large enough or if the NN is too complex. This way, the embeddings are learned in a way that is not able to generalize and make predictions over unseen data, since the latent space representation is too specific. Different techniques to prevent overfitting can be used. A way to notice it is to use methods like cross-validation and early stopping: this takes into account the error on both training and validation set and stops when the number of epochs after which there was no improvement over the selected metric is reached, even if the value of the training error was decreasing.

Backpropagation suffers the vanishing problem issue. What is it and what is it caused by?

Vanishing gradient is an issue in NN that happens during gradient descent and backpropagation. It is linked to weight initialization but mostly to the choice of the activation function. Functions like Sigmoid and Tanh have their derivatives between 0 and 1, and in saturation zones values can be mapped to very small numbers. This can become an issue in deep NN (and especially RNN) because these values are backpropagated through the network and used to compute the gradients of previous neuron weights to update them. Multiplying a lot of small values leads to having very small gradients that are not able to update the weight efficiently, and thus converge.

Can the choice of the activation functions help with the vanishing gradient issue? Why?

Yes, activation functions can directly influence the vanishing gradient. As I said before, the functions that have a derivative output between 0 and 1 are not ideal when the net is prone to vanishing gradient. A possible solution for this issue can be the ReLU activation: contrary to sigmoid and tanh, its derivative can only assume values of 0 and 1, so the gradient is backpropagated without becoming too small. A possible issue however is the dying neurons: when in the saturation zone (negative inputs), the derivative is always 0. If the gradient is multiplied by 0, its value cannot be used to update the neuron's weight, and thus these neurons become dead, not receiving any update. Using the variant of Leaky ReLU solves this issue.

Can the choice of the weight initialization help with the vanishing gradient issue? Why?

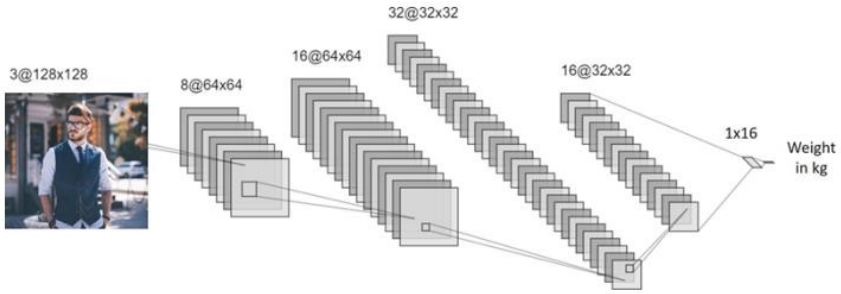
Yes, it can help. Vanishing gradient can be linked to a weight initialization that uses small weights from the beginning. To avoid this, we can use initializations such as Glorot or Xavier, which sample the weight values from a gaussian distribution having the variance that's proportional to the number of inputs. This way, we can keep it under control and have more stability and fast convergence.

Can the use of dropout help with the vanishing gradient issue? Why?

No, dropout is not used to prevent vanishing gradient. Instead, it can be useful when dealing with overfitting: that's because it works by randomly selecting some neurons to "turn off", and this way force the net to learn redundant features of the dataset and be more robust and generalizing. This way, is like having different subnets that are averaged to find the final structure.

Can transfer learning help with the vanishing gradient issue? Why?

Yes, transfer learning can help dealing with vanishing gradients because it exploits a previously trained model that's guaranteed to be efficient. Its weights have been selected during its original training to avoid this kind of issues, so it's reliable. This way, we only need to train the last layers to adapt the problem to a specific objective. It also makes the trainable part "less deep", so less vanishing chances.



What task is addressing the network illustrated above? What loss would you use for training?

How would you modify the network to also predict whether the human is an engineer or not? How would you train the network in this latter case?

It looks to be a regression problem, specifically the task of estimating the weight of a person through a CNN with 4 Conv layers and a final GAP layer. For this objective a loss like MSE can be a wise choice. This can be done I THINK (secondo spunti di co-training da recsys e RCNN) using the feature extraction part for getting a robust latent representation and use it in parallel to compute the regression task and the classification task outputs. The classification will use another layer and a final binary crossentropy loss function. The outcomes of these 2 losses can then be used (even using different weights for the different sections) to obtain a loss function that's a linear combination of the two, and will be used for training the net, much like in object localization tasks (RCNN etc.).

Enumerate the building blocks of the above network as if you were going to implement it in Keras. Consider that there might be different viable options in terms of layers type and parameters. You can choose the one you prefer, but you need to: 1) report any assumption you make 2) include *all* the layers and for each of them report - the layer type - the input and output sizes - the formula used to compute the number of parameters in the layer, e.g., 3x5x5=45 and not just 45. List each layer of the network in a different row of your answer to improve readability.

Input(InputLayer)	(None, [128 128 3])	[0]
conv1 (Conv2D)	(None, [64 64 8])	[8(2x2x3+1)]
conv2 (Conv2D)	(None, [64 64 16])	[16(1x1x8+1)]
conv3 (Conv2D)	(None, [32 32 32])	[32(2x2x16+1)]
conv4(Conv2D)	(None, [32 32 16])	[16(1x1x32+1)]
avg_pool(AveragePooling2D)	(None, [1 1 16])	0
Output(Dense)	(None, [1 1 1])	1(16+1) ??

A) Illustrate the major differences between a classification and an object detection network, and what are the major advantages of these latter over baselines built upon a CNN for classification. Consider R-CNN as a reference for object detection network.

B) Describe what are the major changes introduced by Fast R-CNN and Faster R-CNN with respect to the baseline R-CNN.

A)

A classification network is usually thought to have a fixed structure of the inputs, thus it can be a CNN with FC final layers. These layers are necessary for classification since they can combine the extracted features and extract a probability distribution over the classes, but make the structure less flexible. In addition, classification networks do not need to retain any spatial information to generate their outcome, so the operation of reducing the feature maps to a vector does not lose relevant information. They only need to be trained with crossentropy to maximize the predictions.

Networks for detection like RCNN need to also consider the bounding box coordinates of an object. They include a region proposal section that is in charge of finding the most likely areas in a picture to contain some objects. This is done through a separate network. These predictions are resized to a fixed dimension (because of the presence of FC layers in the CNN) and fed to a standard CNN that was trained to execute the relevant task. The final outcomes are given by a SVM for classification and a BB regressor to estimate the coordinates of the bounding boxes, so the loss function has to take into account a classification loss and a regression loss.

B)

If we consider RCNN, we can see that having an external RPN and a CNN that are separate is inefficient, because training is inefficient and extracted features are not reused. Fast RCNN uses another CNN to process the image. The candidate BBs are directly taken from the extracted feature maps to reuse the convolution outcomes, since they can keep some spatial information. These images are then sent to ROI pooling layers that extract a feature vector for each region proposal and sent it through FC layers that estimates both classification (softmax) and BB coordinates. It however does not integrate the BB extraction into the training since it is performed in a fixed manner, even if the whole structure is trainable end-to-end.

Faster RCNN uses the feature maps extracted from the initial CNN to generate region proposals using a RPN, which is a network trained for the specific task. It uses a sliding window to get sections from the feature maps and feeds them to a FC section for both classification and BB proposal, then these proposals are resized to fixed dimensions and sent to FC layers for BB and class estimation. The training loss is now calculated over 2 regressions and 2 classification losses.

When we could prefer the use of Recurrent Neural networks instead of Long Short-Term Memory networks? Why?

We could prefer RNN to LSTM when handling short data sequences and NN that are not very deep, because these structures suffer from vanishing gradient issues but are generally less prone to overfitting and have a fewer number of parameters to learn. On the other hand, LSTMs are preferred when sequences are long, because thanks to the CEC they remove the vanishing gradient issue, but are more expensive in terms of training.

Describe briefly the seq2seq model. Can it be used with RNN or it can only be used with LSTM? Why?

The seq2seq model is used to work with data sequences. It is able to take into account past inputs to generate the outcomes. It follows the encoder-decoder architecture, using the first part to generate a robust latent representation of the sequence and then training a decoder according to the information extracted, mapping a sequence of inputs to a sequence of outputs. They are trained using pairs of <target, context> to estimate the next element in a sequence given the previous ones, called context. The correct sequence is used by the decoder during training and then at inference the decoder uses the past outputs to generate its prediction.

They are usually implemented using LSTMs because of the relevant use of memory they need, processing long sequences of words, but can be used with RNN when the structure is not deep and the sequence short.

25/01/2021

Let's start from the core issue of machine learning, i.e, the overfitting/generalization trade-off. Discuss if and how deep learning (as a general paradigm) differs from (classical) machine learning regarding the overfitting issue.

Hints: start by defining the concepts, then highlight the differences, then connect if/how these differences are related to overfitting/generalization ...

Deep Learning is a part of machine learning. This is why ML is defined as the discipline that uses algorithms to search for patterns in data and extract features from them, to learn something about its characteristics. Given a task T, it can be optimized through experience E given a metric M. The key difference is that ML does this by manually extracting the interested features while DL uses neural networks to perform both the extraction and the task. Overfitting happens when the algorithm gets too used to the data shown while training and is unable to perform correctly on unseen data. It can happen when the training set is scarce or the net is too large with too many parameters. Because of this, DL models may be more prone to overfitting because they require more data to be trained and are usually larger, while ML models suffer from underfitting because they don't usually have many parameters.

Does deep learning work only for images and text? Motivate your answer.

No, DL can also be used with things like time series, audio tracks etc, since their structure is very general and able to learn patterns in datasets without caring about what they are. It only matters that they're turned into a meaningful numerical representation and that the NN structure is coherent with the operation it needs to perform on the specific dataset.

What are the major differences when training feed forward neural networks for regression and for classification? Motivate your answer.

Classification and regression are two problems that can be solved by NN. The first one involves assigning a label to an input from a fixed set, the second one assigns a scalar value to the input. They differ because they need different activations and loss functions to perform optimally. For example, activations like Sigmoid and Tanh are more indicated for classification, since they can map their outputs in an interval that

can be used to generate a probability distribution over classes. For regression, since we want to estimate a number, linear or ReLU are more commonly used. Loss functions are the crossentropy for classification, since we need to verify if the correct label was applied to the input, and MSE for the regression, to evaluate how different the values were from the correct ones.

List and discuss briefly three (3) techniques used to limit overfitting in neural networks.

Overfitting can be limited using different approaches. Some examples are:

- Cross-Validation and Early Stopping: a section of the dataset is used as validation set. This set is used to estimate the generalization capabilities of the network during training. The error is computed both on the training set and on the validation set, but this last one is not used as input for the NN. Early stopping uses metrics to find out when the network is starting to overfit, given that there were no improvements on the selected metric (like validation error) for some epochs, and stops the training even if the training loss was still decreasing;
- Dropout, that "turns off" some neurons according to a certain probability so that they are not updated and thus the net is forced to learn redundant data representation, becoming more robust. This way there are a series of "subnets" that are all averaged together to make the final one;
- Weight Initialization, because a correct initialization is pivotal for a correct learning of the net. Using large weights can be dangerous because it may lead to overfit, while smaller weights are preferred but in deep NN they suffer from vanishing gradient issues. This is why techniques like Xaver or Glorot initialization are more indicated;
- Weight Decay, which adds a penalty to the loss function that's proportional to the L2 norm of the weights to keep it under control and make so that they do not assume too large values. It uses a gamma parameter to assign importance to the regularization term;
- Batch Normalization, which standardizes each layers output considering the mean and variance of the activations, so that they are scaled accordingly. It also uses 2 parameters beta and gamma to tune this transformation;

What is it and how can you tune the gamma parameter of weight decay?

In WD, the gamma parameter is used to give importance to the regularization factor. This parameter can be tuned by checking the performance for each value of gamma that's selected and then compare them to see the optimal value, using a metric like cross-validation error. Tools like Keras Tuner are fit for this purpose since they iterate over an interval of values and select the most promising one.

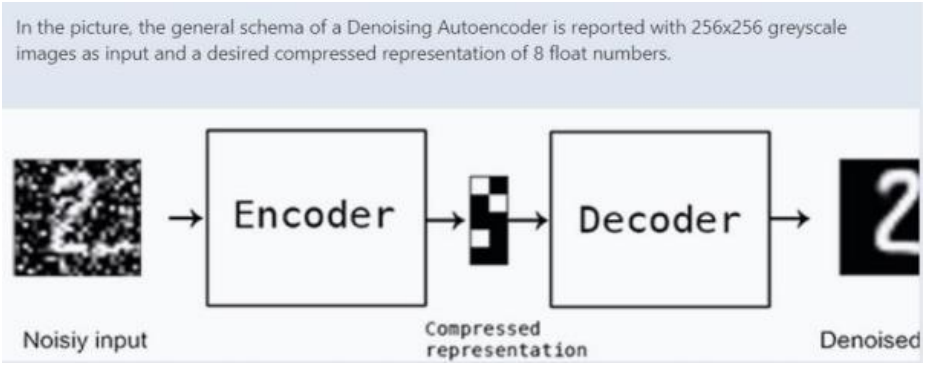
How do you initialize the weights of a deep neural network? Why?

Weight initialization is necessary to make the NN learn in an efficient way. This is because there can be different solutions with different outcomes:

- All 0s initialization: bad idea because this way all weights learn the same values and the net is uncapable of extracting any feature;
- Big values, bad idea because large values make the net more prone to overfitting, and may also be closer to saturation zones of some activation functions and thus cause vanishing gradient;
- Small values, like values sampled from a 0-centered gaussian distribution with unitary variance, ok for small nets but this too may lead to vanishing issues in deeper ones;

- Xavier or Glorot initialization, optimal solution since they use a gaussian distribution that's 0-centered but with a variance that's related to the number of input values;
- Transfer Learning, we can use a pre-trained NN to keep its weights and then modify only the FC layers, since they're not general purpose like the feature extractions.
- Autoencoders, the encoder can provide a good initialization when few data are available in the set.

AUTOENCODER CNN



Conv2D(128,(3,3))	3x3x128+128 = 1280	256,256,128
MaxPooling2D((2,2))	0	128,128,128
Conv2D(64,(3,3))	3x3x128x64+64=73792	128,128,64
MP2D((2,2))	0	64,64,64
Conv2D(32,(3,3))	3x3x64x32+32	64,64,32
MP((2,2))	0	32,32,32
Conv2D(16,(3,3))	3x3x32x16+16	32,32,16
MP((2,2))	0	16,16,16
Flatten()	0	16x16x16 = 1,1,4096
Dense(32)	32+32x4096=131104	1,1,32
Dense(8)	8+8x32 = 264	1,1,8
Dense(32)	32+32x8=288	1,1,32
Dense(4096)	4096+32x4096=135168	1,1,4096
Reshape((16,16,16))	0	16,16,16
Conv2D(16(3,3))	2320	16,16,16
UpSampling2D((2,2))	0	32.32,16
Ecc ecc fino ad arrivare alla size originale		
Output(1,(3,3))	…	256,256,1

Describe the training procedure for the Denoising Autoencoder in the picture in terms of the dataset you need and the loss function you would use.

Autoencoders can be used for denoising. They can be fed as input a dataset of images in which some noise was added, to be able to learn more robust latent representations that only capture the most meaningful features of the input. The loss function used for training can be any loss that measures the difference between the reconstructed output and the input, like MSE ||s-D(E(s))||_2.

What kind of applications can be solved with a Recurrent Neural Network? Make two (2) examples and discuss their characteristics.

A RNN is a net that is able to process sequential data because it is able to retain some memory of previous inputs, and use them to generate the following outputs. RNN are composed of a context network and of a standard FFNN that also uses the outputs of the CN to estimate the outcome of its neurons. The context network keeps track of the temporal evolution of the net, and is unrolled to have each layer corresponding to a timestep, so that these layers are replicas of the state of the net and use the same weights and biases. RNN can be used as an example for time series forecasting, that is predicting a certain value in a sequence of numbers, or language processing, like translations, because they're capable of extracting the relationship between the input data and contextualize them in a specific order.

Recurrent Neural Networks can suffer the Vanishing Gradient problem. Discuss what it is due to and how to solve it. Does this problem exist in feed forward neural networks?

The vanishing gradient is an issue that is present in RNN. This is due to the particular structure of them that favors the conditions for it to happen. This is because VG is due to the use of certain weight initialization but mostly because of some activation functions in the neurons, like Sigmoid and Tanh. These functions have their derivatives mapped in an interval between 0 and 1: in RNN, layers represent timesteps, so they all have the same parameters. This means that the same derivatives are multiplied at each timestep when performing BPTT. Given that these values are very small, deep RNN have their gradients become very small in the first layers. This problem can be solved by using a different activation, like ReLU, that however may suffer from dying neurons, or using LSTM structures to perform the same tasks.

FFNN also may have vanishing gradients, for the same reasons as before, but if they're not particularly deep and the initialization is performed correctly (also adding some techniques like BN helps) the problem is less frequent than in RNN.

10/02/2020

With reference to Feed Forward Neural Networks answer the following questions:

1. What is the difference between supervised and unsupervised learning?

2. Make an example of neural network used for supervised learning, i.e., describe the problem faced, the model, and the loss function used

3. Make an example of neural network used for unsupervised learning, i.e., describe the problem faced, the model and the loss function used

1. In supervised learning, the training phase of the NN is performed using a set that's annotated with the desired output. This way, the NN can learn how to generate the correct output because this information is given to it, and the net can extract the relevant characteristics to associate features to outputs. In unsupervised learning instead, the NN has no information on the desired output, and it extracts features autonomously by examining the dataset and noticing some patterns and characteristics to make predictions.

2. For supervised learning, a possible example is a CNN for classification of images. Given a training set of annotated images, the NN tries to extract features to make associations with the labels provided, so that it is able to classify unseen data. It uses multiple Conv layers to identify features and some final FC layers to combine them and make a prediction using a softmax distribution. A common loss function is the categorical crossentropy, that measure the distance between the prediction and the correct label.

3. For unsupervised learning, a model can be the Neural Autoencoder. An autoencoder is a kind of NN that tries to reconstruct the input, learning the identity function. This is done through an architecture that includes an encoding part, in which the dimensionality of the input is reduced to generate a smaller representation that keeps only the most relevant information, and a decoder, that starts from the representation and tries to reconstruct the input. The autoencoder is trained over the distance of the reconstructed image from the original image, using a function like MSE: ||s-D(E(s))||_2, so that it does not need any additional labeling.

Many architectural details are involved in obtaining a successful neural network model. For each of the following, list and discuss briefly the available options:

1. Output activation function

2. Loss function

3. Weight initialization

4. Regularization

1. Activation functions can be used to introduce non-linearities in the NN. They are important because they provide a more extensive feature extraction and combination process. The choice depends on the kind of task that we want to solve and on the net's structure. As an example, Sigmoid, Softmax and Tanh are indicated for classification problem, because they can map values in an interval used to generate a probability distribution, while ReLU is used for regression tasks.

2. Loss function also depends on the task that we need to solve. Classification problems use binary or categorical crossentropy according to the kind of labeling they need to perform, while regression problems use functions like MSE and MAE, also we've seen that in reconstruction loss autoencoders use MSE.

3. Weight initialization is necessary to make the NN learn in an efficient way. This is because there can be different solutions with different outcomes:

- All 0s initialization: bad idea because this way all weights learn the same values and the net is uncapable of extracting any feature;
- Big values, bad idea because large values make the net more prone to overfitting, and may also be closer to saturation zones of some activation functions and thus cause vanishing gradient;
- Small values, like values sampled from a 0-centered gaussian distribution with unitary variance, ok for small nets but this too may lead to vanishing issues in deeper ones;
- Xavier or Glorot initialization, optimal solution since they use a gaussian distribution that's 0-centered but with a variance that's related to the number of input values;
- Transfer Learning, we can use a pre-trained NN to keep its weights and then modify only the FC layers, since they're not general purpose like the feature extractions.
- Autoencoders, the encoder can provide a good initialization when few data are available in the set.

4. Regularization can be performed to make the input data more structured and speeding up the convergence. It is also used to make so that the weights in the net do not assume values that are too large and lead to overfitting. Regularization can be performed by adding to the loss function the norm-1 or norm-2 of the weights, times a gamma factor that's a hyperparameter and is needed to balance the importance of it.

Parametric Search CNN

```
import keras
from keras.utils  import np_utils
from keras.models import Sequential, Model
from keras.layers import Dense, Activation, Flatten, Conv2D,
from keras.layers import MaxPooling2D, BatchNormalization

pool_size          = (2,2) # size of pooling area for max pooling
nb_filters         = 64

model = Sequential()

model.add(Conv2D(nb_filters, (11,11), input_shape=(64, 64, 3), padding = "same"))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size = pool_size))
model.add(Conv2D(nb_filters*2, (5,5), padding = "same")) #2nd conv Layer starts
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size = pool_size))
model.add(Conv2D(nb_filters*4, (3,3), padding = "same")) #3rd conv Layer starts
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size = pool_size))
model.add(Conv2D(nb_filters, (1,1), padding = "same"))   #4th conv Layer starts
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size = pool_size))
model.add(Flatten())
model.add(Dense(64))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dense(16))
model.add(Activation('relu'))
model.add(Dense(21))
model.add(Activation('softmax'))

model.summary()
```

1. Is this a classification or a regression network? How many outputs?

2. The following is the output of the model.summary() command. A few numbers have been replaced by dots. Please, fill all the numbers in and in particular indicate which computation you are performing to get to each number. Note: the first dimension is the batch size, and as such this is set to None.

3. How big is the receptive field of the pixel at the center of the feature map after the first convolutional layer? How about the pixel at the center of the second convolutional layer?

4. This network has relatively large filters in the first layers. Can we replace that layer with others to reduce the number of parameters and at the same time preserve (or increase) the receptive field?

5. What happens if we remove the fourth convolutional layer: does the number of parameters increase or decrease? Any comments about that layer?

1. Classification network, non-linear model (activations like relu), used to output through a softmax a probability distribution over 21 classes.

2. See tables below.

3. RF after first conv layer is 11x11, after second conv layer is:

1 -> 5+(1-1) = 5 -> 5*2 = 10 -> 11-(10-1) -> 20 x 20

4. Assume we can replace only the first layer, which has a filter of size 11x11. We can replace the first conv layer and place 2 blocks of conv 5x5 + maxpool 3x3. in this way we reduced the number of parameters (in total we have 64 * (5*5*3+1) parameters * 3 conv layers, instead of the 64 * (11*11*3+1) of the initial conv layer). The receptive field after these layers is the same, because after the 3 conv5x5 and maxpool3x3 becomes = 11x11. Note: other possible answers are possible. We could reduce the filter size to 3x3 so to have less parameters but increase the size of max pooling to 7x7 for example. In fact, max pooling has no parameters but increasing its size, increases the receptive field.

5. If the fourth convolutional layer is removed, the number of parameters in the network will decrease. The fourth convolutional layer is responsible for reducing the spatial dimensions of the feature maps produced by the previous layer, so its removal will result in larger feature maps in the output of the third convolutional layer. However, this removal of the fourth convolutional layer may lead to overfitting, as the network will have more capacity to memorize the training data. This is because the fourth convolutional layer acts as a regularization layer by reducing the size of the feature maps, which reduces the capacity of the network.

Conv2D	(64, 64, 64)	(11 * 11 * 3 + 1) * 64
ReLU	(64, 64, 64)	0
MaxPooling2D	(32, 32, 64)	0
Conv2D	(32, 32, 128)	(5 * 5 * 64 + 1) * 128
ReLU	(32, 32, 128)	0
MaxPooling2D	(16, 16, 128)	0
Conv2D	(16, 16, 256)	(3 * 3 * 128 + 1) * 256
ReLU	(16, 16, 256)	0
MaxPooling2D	(8, 8, 256)	0
Conv2D	(8, 8, 64)	(1 * 1 * 256 + 1) * 64
ReLU	(8, 8, 64)	0
MaxPooling2D	(4, 4, 64)	0
Flatten	(1024,)	0
Dense	(64,)	1024 * 64 + 64
BatchNormalization	(64,)	128
ReLU	(64,)	0
Dense	(16,)	64 * 16 + 16
ReLU	(16,)	0
Dense	(21,)	21 * 16 + 21

Text is a challenging domain where several challenges need to be faced to learn successful models. With reference to machine learning on text data answer the following questions:

1. Text requires to be encoded, describe what is word embedding, what it is used for, and how it is obtained with the word2vec model.

2. Text comes in sequences, describe the Long Short-Term Memory cell and the architectures which can be used when inference is done online, i.e., one word at the time, and batch, i.e., when the entire sentence is available.

3. Long Short-Term Memory cells solve the vanishing/exploding gradient problem of Recurrent Neural Networks. What is this problem due to? How do they solve it?

1. It is necessary to turn text in a numerical form because otherwise it cannot be processed by a NN. Initially, 1-hot encoding was used, but it suffered from dimensionality problems, and it was uncapable of capturing meaningful relationships between word semantics. Word embedding is a technique that maps the words in a space with smaller dimensions, also capturing the similarities in their meanings. It is used for tasks that require language processing, like translations and predictions. In the word2vec model, it is obtained through a projection of the 1-hot encodings into the embedding space, allowing the net to learn a correct representation through the input of <target, context> pairs, thanks to which we can have information about the links between words.

2. LSTM is a type of RNN used to process sequential data in a way that's able to keep a memory of past outputs, and generate new values according to them. It is implemented using different kind of gates and linear activations. The cell has a forget, input, output and memory gate, each one used for a different task (deciding the long-term memory to pass forward, the input and short-term memory, outputting the final computation, combining the ling-term and short-term values). When inference is done online, i.e., one word at a time, the most common architecture is the One-to-One LSTM architecture. Each time step in the input sequence corresponds to a single time step in the LSTM layer, and the LSTM layer processes the sequence one word at a time. The final state of the LSTM layer is then used as the feature representation for the entire sequence, and it is fed into a feedforward network for prediction. When inference is done using a batch, i.e., when the entire sequence is available, the most common architecture is the Many-to-Many LSTM architecture. In this architecture, the entire sequence is input into the LSTM layer, and the LSTM layer outputs a sequence of hidden states, one for each time step in the input sequence. These hidden states can then be used to make predictions at each time step in the sequence.

3. LSTM can solve the vanishing gradient problem that's an issue in RNN. This is because the VG is linked to the activations used in the layers: if these activations have their derivatives in a small interval, like Tanh and Sigmoid that have theirs between 0 and 1, these values are multiplied for the computations in each timestep, given that they're the same since BPTT uses the same weights because of the unrolling. This can lead deep nets to have very small gradients for their earlier layers. Exploding gradient can happen when the values that are multiplied are big. LSTM solve this problem by using the CEC, which is used to fix the error of the network to 1, and using linear activations in the layers. The net is still able to learn efficiently because of the gated structure that can decide how much to keep and discard from the past inputs.

15/01/2020

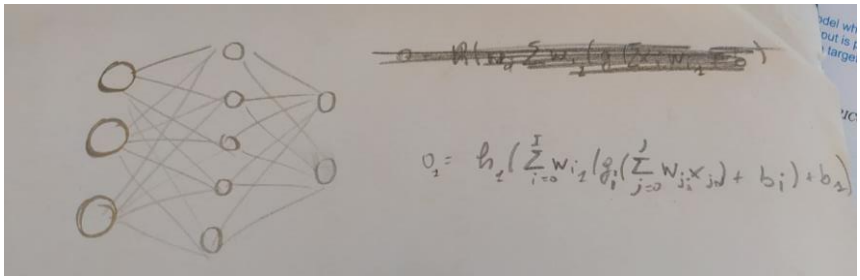
With reference to a Feed Forward Neural Network with 3 input, 1 hidden layer having 5 hidden neurons, and an output layer with 2 neurons.

1. Draw the network, and provide its output characteristics, i.e., the mathematical formula, for the output of the previous network as a function of its input and weights

2. Consider the previous network to be used for classification. Define its activation functions and error/loss function providing motivations for your choices

3. What is backpropagation and how does it work?

1.



2. If the net is used for classification, it may be a good idea to use activation functions like Sigmoid or Tanh. It also seems to be a binary classification problem, so they should be fitting. As loss function, the most indicated one is the binary crossentropy, that tries to maximize the correct predictions over the two possible classes.

3. Backpropagation is the process used in NN training when performing gradient descent. Gradient descent involves using the derivatives of the loss function on the output of the net with respect to each weight to determine the direction in which the error can decrease the most. This way, using the chain rule, we can use this gradient to estimate each weight, computing the derivatives. To do this, the outputs of the previous layers' activations are necessary to find the gradient of the previous layers. In this sense, the error is backpropagated through the network, keeping the directional information, because older layers use the newer layer's derivatives when finding the gradient with respect to their parameters.

When training neural networks, we need to use some form of regularization. Answer in detail the two following questions about regularization

1. What is regularization and why do we need regularization when training an artificial neural network

2. Describe TWO (2) regularization techniques used in training artificial neural networks providing their rationale/motivation, i.e., what do they do and why they do that.

1. Regularization is a technique used in NNs when performing gradient descent. The weight values need to be set correctly, otherwise the training is inefficient, and it can lead to overfitting or underfitting. If the weights assume great values, the generalization capabilities are lost. Because of this, a regularization term is added to the loss function.

2. Techniques used are L1 and L2 regularization: L1 adds to the loss function a term that's the norm-1 of the weights, and L2 adds the norm-2 of them. Both of these terms are weighted by a gamma factor, which is a hyperparameter of the net that needs to be tuned for optimal performance.

Answer the following questions providing a short explanation for each one.

1. Is this a linear model?

2. Is this a classification or a regression network?

3. What is the input size? What is the output size?

4. Is this a fully convolutional neural network?

5. Once trained, can we feed to the network a larger image? Can we feed an image with more channels? Why?

6. Is there any form of regularization?

7. The following is the output of the model.summary() command. A few numbers have been replaced by dots. Please, fill all the numbers in, together with the computation you are performing. Note: the first dimension is the batch size, and as such this is set to None.

```
import keras
from keras.utils  import np_utils
from keras.models import Sequential, Model
from keras.layers import Dense, Activation, Flatten, GlobalAveragePooling2D, Dropout
from keras.layers import Conv2D, MaxPooling2D, AveragePooling2D, Dropout
from keras.applications import MobileNet, VGG16

patchsize          = 64
pool_size          = (2,2)
kernel_size        = (3,3)
nb_filters         = 8

model = Sequential()

model.add(Conv2D(nb_filters,  (3,3)  , input_shape=(patchsize, patchsize, 3), padding
= "same"))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size = pool_size))
model.add(Conv2D(nb_filters*2, (3,3), padding = "same"))
model.add(Activation('relu'))
model.add(Conv2D(nb_filters*2, (3,3), padding = "same"))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size = pool_size))
model.add(Conv2D(nb_filters*3, (5,5), padding = "same"))
model.add(Activation('relu'))
model.add(Conv2D(nb_filters*3, (5,5), padding = "same"))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size = pool_size))
model.add(GlobalAveragePooling2D())
model.add(Dense(64))
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(16))
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(5))
model.add(Activation('softmax'))
```

1. No, this is not a linear model since it contains nonlinear activation layers (relu);

2. This seems to be a classification network since the final layer uses the softmax activation, which is generally used to output probability distributions over classes;

3. From the layers we can see that the input size is (64, 64, 3) and the output size is (5,);

4. This is not a FCNN because it uses Dense layers, which are not convolutional layers but fully connected;

5. Yes because of the presence of the GAP layer that is used to output a single value per feature map, so the height and width of the maps do not matter! However, depth has to be kept, because the FC layers need to know the dimension of their inputs, in this case 24, to define their weight matrix;

6. Yes, there's two Dropout layers with a 0.5 parameter which are used to regularize the model in order to prevent overfitting, by switching off some neurons and make the net learn a more robust representation of the inputs.

7.

Conv2D	(64, 64, 8)	(3 * 3 * 3 + 1) * 8
ReLU	(64, 64, 8)	0
MaxPooling2D	(32, 32, 8)	0
Conv2D	(32, 32, 16)	(3 * 3 * 8 + 1) * 16
ReLU	(32, 32, 16)	0
Conv2D	(32, 32, 16)	(3 * 3 * 16 + 1) * 16
ReLU	(32, 32, 16)	0
MaxPooling2D	(16, 16, 16)	0
Conv2D	(16, 16, 24)	(5 * 5 * 16 + 1) * 24
ReLU	(16, 16, 24)	0
Conv2D	(16, 16, 24)	(5 * 5 * 24 + 1) * 24
ReLU	(16, 16, 24)	0
MaxPooling2D	(8, 8, 24)	0
Average_pooling2D	(1, 1, 24)	0
Dense	(64,)	24 * 64 + 64
ReLU	(64,)	0
Dropout	(64,)	0
Dense	(16,)	64 * 16 + 16
ReLU	(16,)	0
Dropout	(16,)	0
Dense	(5,)	16 * 5 + 5
Softmax	(5,)	0

Consider the problem of sequence modeling and the classical Seq2Seq model used for machine translation. Answer the following questions:

1. What is "sequence to sequence modeling"?

2. Describe the Long Short-Term Memory cell and its use in sequence modeling

3. Describe the classical seq2seq model based on LSTM cells for language translation, its training procedure and its run-time inference

4. What is an attention mechanism? How can it be used in a Seq2Seq model?

1. Sequence to sequence modeling is a structure of NN that is used to work with sequences of data. It uses an architecture that's able to preserve information about the past to generate new outputs. Their goal is to map the input sequence in another one. There can be different tasks interested when talking about this modeling, the most common ones are language translation (many-to-many).

2. LSTM is a type of RNN that is able to keep memory of past data. It achieves this using a cell structure, that has inside different kinds of gates that use sigmoid and tanh function to be combined:

- Input gate: it is used to combine the input and the previous memorized short-term memory value;
- Forget gate: used to determine how much of the long-term memory to keep;
- Memory gate: combines the forget gate and the input gate's outcomes to determine the new long-term memory;
- Output gate: uses the values of the memory gate, the input and the short-term memory to generate the new short-term memory.

LSTM is also able to eliminate gradient vanishing and exploding issues thanks to the CEC.

3. Seq2seq architectures use LSTM memory cells to realize an encoder-decoder structure. This is used to generate a compact representation of the input so that the output can be a new sequence generated considering the relevant feature extracted by the encoder. The input is given in the form of pairs <sequence, target> where we want to estimate a new sequence given the initial one, maximizing the crossentropy. During training, learning is performed, also in the decoder, using the correct sequence values as input, so that it can learn now to process them and the existing relationships, while at inference the decoder generates the new sequence by using the values computed by the previous cells.

4. Attention is used in seq2seq models because it allows them to give different relevance to different sections of the input sequence. This way, the learning is more efficient because the more important parts for prediction are highlighted. It is performed by computing some weights through which weighting the latent representation generated by the encoder. These weights are found considering the current decoder state and the hidden states of the encoder, and then generating a distribution. The sequence is weighted by these values, and the output is generated also using this information.

19/6/2020

Which conceptual difference does make Deep Learning differ significantly from being just another paradigm of Machine Learning similarly to supervised learning, unsupervised learning, reinforcement learning, etc.?

Deep learning is still a branch of machine learning, in the sense that given an algorithm with a task T and some experience E, through some metric M we are able to evaluate the performance of the algorithm, and improve its performance. They both process datasets and try to get their most prominent characteristics to best describe them and be able to make predictions. However, the processing is done through features that in machine learning are manually extracted and selected by the operator of the model through knowledge or experience, while in DL models are also used to extract them. These models also differ in their structures, since DL uses deep neural networks that have multiple layers of processing units. Thanks to this, they're able to capture hidden and more complex relationships in the dataset.

Make an example of an application where Classical SUPERVISED learning is used and then present its Deep counterpart. -> (I want it a real, short. fully specified, application example, including the algorithms and the models ... there should not be another answer like your in the class).

Classical supervised learning can be represented by logistic regression, which is used for binary classification problems. However, even if it can be simple and performing on specific datasets, it is not able to capture nonlinear relationships well. Its deep learning counterpart can be a CNN for binary classification, which uses multiple convolutional layers to extract features and then output a binary probability distribution using an activation function like sigmoid.

Make an example of an application where Classical UNSUPERVISED learning is used and than present its Deep counterpart. -> (I want it a real, short. fully specified, application example, including the algorithms and the models ... there should not be another answer like your in the class).

An example of unsupervised ML can be found in K-Means clustering, where a dataset is processed and the algorithm is able to extract some groups of elements in an iterative way through the use of centroids. Its deep counterpart may be represented by an autoencoder: these models are trained in an unsupervised way to recognize features and reconstruct the input through them. This way, they can also be used to encode similar elements in representation that are closer in the latent space, forming groups.

What is the relationship we learn in a neural autoencoder? Why we do it?

A neural autoencoder is a NN model that has the aim of learning the identity function. This is because in the decoder section it tries to reconstruct the input using only the most relevant features extracted in the encoding section, to learn a lower dimensional representation of it that only takes into account the important information. It learns the mapping of the input in a low dimensional latent space and is able to also find the inverse mapping. This way, they can be used for multiple applications like weight initialization, noise reduction, anomaly detection.

How could we size the embedding of a neural autoencoder?

Usually, the number of layers and neurons is a hyperparameter that needs to be found for optimal performance. This value can be found through trial and error, through tuning, evaluating the outputs in relation to it, or by having particular knowledge around the dataset. Cost in terms of memory and time are also to be taken into account.

When would you prefer weight decay with respect to early stopping? How can you tune the gamma parameter of weight decay?

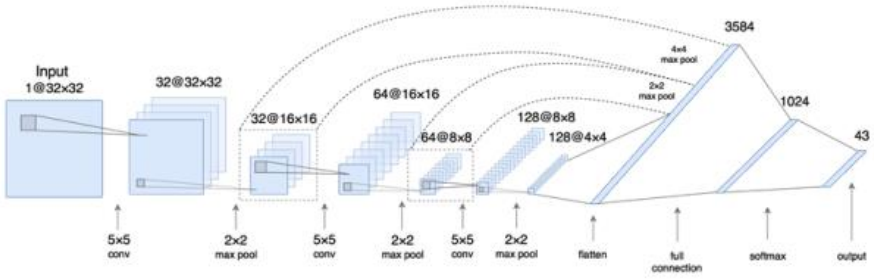
Weight decay and early stopping are two techniques using to prevent overfitting in NN, which happens when the dataset gets too adapted to the training data and loses generalization capabilities. Weight decay is performed by adding to the loss function a term that's found by making an a-priori assumption over the distribution of the weights. This way, their norm is reduced not to make them assume too large values. Early stopping uses a selected metric (often the validation loss) to stop the training, to prevent it getting unnecessarily adapted to the training dataset. Since early stopping usually requires a validation set for the evaluation of the NN's performance, it's preferred when having more data, while weight decay can be useful even with smaller datasets because it does not need a big data split.

The gamma is a hyperparameter of the weight decay that weights the importance of the correction factor. It needs to be selected in the way that maximizes performance and does not impact it. It can be found through hyperparameter tuning, evaluating the loss of the NN considering different values.

Discuss the good and the bad of sigmoid, hyperbolic tangent, and ReLu. When you should use each of them? Why? Provide their derivatives.

Sigmoid, Tanh and ReLU are all activation functions used to introduce nonlinearities in NNs. They are important because they allow the net to capture and combine more complex relationships. According to the specific task, they may be more or less appropriate. Sigmoid and Tanh are more indicated when performing classification tasks, since their output can be used to obtain a probability distribution over classes. They have smooth derivatives that are easy to compute. However not appropriate for deep NNs, since their derivatives have values between 0 and 1, and during backpropagation this can lead to vanishing gradients. ReLU on the other hand is suitable for regression tasks, since it's unbounded, and removes the vanishing gradient problem, having the derivatives assume values of 0 or 1, but it may lead to dying neurons because inputs in a certain range generate 0 derivatives, and thus the weights are not updated.

Parametric Search

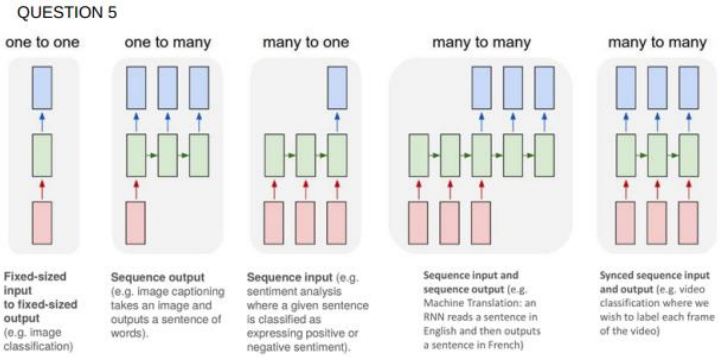


Input	(32, 32, 1)	0
Conv2D	(32, 32, 32)	(5 * 5 * 1 + 1) * 32
MaxPooling2D	(16, 16, 32)	0
Conv2D	(16, 16, 64)	(5 * 5 * 32 + 1) * 64
MaxPooling2D	(8, 8, 64)	0
Conv2D	(8, 8, 128)	(5 * 5 * 64 + 1) * 128
MaxPooling2D	(4, 4, 128)	0
MaxPooling2D	(4, 4, 64)	0 **skip connection**
MaxPooling2D	(4, 4, 32)	0 **skip connection**
Concat di tutti e 3	(4,4,224)	0
Flatten	(3584,)	0
Dense	(1024,)	1024 * 3584 + 1024
Dense	(43,)	1024 * 43 + 43

Describe the network in the IMAGE in terms of the characteristic elements composing it, the rationale behind the architecture, the task it might be supposed to do, the loss function you would use to train it. Justify all your statements.

There are a set of convolutional layers all followed by MP layers. These layers are used to extract features and generate a smaller representation of the input that takes them into account. The net also uses skip connections, concatenating the outputs of previous layers and then flattening them. This vector is then sent to some FC layers for processing. This architecture may represent a classification network that takes as

input a greyscale image and classifies it in 43 classes, and for this reason an appropriate loss function can be the categorical crossentropy.



For each of the models in the IMAGE above provide its description and make an example of it.

1. Classification of images, it uses a CNN to process an image. The input is an annotated image from a dataset and the output is a probability distribution over the desired classes;
2. Image captioning is generated by getting an input image and extracting some features (maybe using an encoding type of structure), and then from these a sequence of words that describe it is generated. This kind of network often uses attention mechanisms;
3. Sentiment analysis comes from the input of a sequence of words through which the net is able to generate a more compact representation using only the relevant information for the task;
4. Models like seq2seq models that use LSTM cells to generate a new output sequence given an input sequence, they have an encoder-decoder structure that allows for latent representation and reconstruction;
5. Video classification performing nets use input sequences and output sequences that are coordinated, and does not need to collect all input for processing the output.

Why do we need an attention mechanism? Aren't Long Short-Term Memories enough?

LSTM are able to keep memory of past outputs and use them to generate new inputs. They do this using a cell structure that has gates and linear activations, also preventing the vanishing and exploding gradient. They however use the given context without any preference: there might be some sections of it that are more relevant for the output's task. Attention uses the hidden states of the network to compute a set of weights that are used to balance the latent representation made by it and used for predictions. This way, the most important information is given more relevance.

Why do we need recurrence mechanism? Isn't attention mechanism enough?

The attention mechanism itself does not provide a way to learn from past context, it's just a way to give different relevant scores to its sections. It needs the recurrence because this way the net can generate outputs using also past information, and these values are the ones considered by the attention when used as inputs for successive layers.

What does the sentence «You shall know a word the company it keeps» by John R. Firth (1957) mean? Why do we mentioned it in the course and which model uses it? Describe the model.

The sentence is referred to language models. These are NNs that are able to process words to make predictions around sentences. They use numerical representations of them to process and extract information. This method uses couples of <target, context>, which are respectively a word and the sentence it's in, to estimate a probability of the target appearing. These representations are usually encoded using 1-Hot vectors or Word Embeddings, which are smaller dimensional representations that also take into account word similarities. A model that uses these is the Word2Vec model, which works with word embeddings to estimate the probability of a word in a context (CBOW), using the information of the context both before and after the word, or of a context given a word (SkipGram).

15/07/2020

Why Deep Learning?

Deep Learning is a subfield of machine learning. Just like it, it involves using an algorithm that, given a task and some examples from experience, is able to be optimized and evaluated through a metric, and can thus learn features regarding a dataset. The difference is in that feature selection is done manually in ML, and computations are usually less complex, while DL for doing this uses a set of nonlinear units called neurons, that are organized in layers to form a structure. These elements are able to extract features autonomously and learn from experience in a meaningful way, providing more significant representations of data since they're able to find relevant characteristics and relationships.

What is overfitting? What it is due to?

Overfitting is a phenomenon that can affect NNs: since they use training data to build their model, it can get too accustomed to local features and lose generalization capabilities, performing well on known data but underperforming on unknown samples. This can happen for several reasons, like a too long training time, a scarce dataset compared to the number of parameters, high values for the weights.

How do you avoid overfitting when training deep neural networks?

Overfitting can be avoided using several techniques:

- Weight Decay, adding a parameter to the loss function which is generated by making an a-priori assumption over the weight distribution to keep their norms under control. This parameter is multiplied by a gamma factor, which has to be tuned to find the optimal value that balances the decay;
- Early Stopping, involves monitoring both the validation error and the training error. It uses a metric to monitor, and according to a patience hyperparameter it determines when to stop the training of the NN. This happens after there were no epochs where the selected metric improved after some time;
- Regularization, the loss function can be given another weighted parameter to make so that the weights are selected in a certain interval and they do not assume big values. This can be done by adding the norm_1 (Lasso) or norm_2 (Ridge) of the weights to it;

- Batch Normalization, the operations of normalization and standardization are commonly used as preprocessing, but here BN layers are introduced to perform this operations also on the values of the hidden layers, considering their average and variance;
- Dropout, involves turning off some neurons according to a certain probability. This way, the net is forced to learn redundant representations of data, thus becoming more immune to random noise and training set peculiarities. In the end, it's like having different NNs that are averaged when making the final model;
- Weight initialization, it's useful to have the weight selected in a way that makes the learning efficient and avoids vanishing/exploding gradients and overfitting. High values and 0 values are not good choices, while small values are suited for small nets, but techniques like Xavier or Glorot initialization, that sample them from a distribution that has a variance proportional to the number of inputs, are a better choice.

What is the pain behind weight initialization? What is the relief?

Weight initialization is an important step when building the NN. Several choices are possible:

- All 0s, not a good choice because doing so all weights are updated in the same way and thus the net is not learning any feature;
- Large values, not good because big values can lead to several issues, like overfitting and vanishing gradients since they're more likely in saturation zones of some activation functions;
- Small values, like samples taken from a 0-centered gaussian distribution with unitary variance, it's a good solution for small nets but in deeper ones having values that small can lead to vanishing gradients.
- Xavier or Glorot initialization, they're optimal choices because they sample the values from gaussian distribution that's 0-ventered but has its variance proportional to the number of inputs;

What is he pain behind vanishing gradient? What is the relief?

Vanishing gradient is ann issue that affects backpropagation: sometimes the gradient becomes very small in the earlier layers and the weights are not updated successfully, affecting the convergence. This may be due to incorrect weight initialization but mostly to the activation functions that are selected: Sigmoid and Tanh have their derivatives between 0 and 1, so when these values are multiplied for all the previous ones the gradient also goes to 0. To avoid this issue, we can use a correct weight initialization, the ReLU activation function, which has its derivatives in 0 or 1, but bay cause dying neurons, sparse connections or LSTM.

What is the pain behind text encoding? What is the relief?

Text encoding is necessary to work with words, since NNs are only able to process numerical values. The easiest way to encode text is using 1-hot encoding, with vectors that are the size of the vocabulary and each words is represented by a 1 in a different position of them. The problem with this representation is that dimensionality is very high, and usually a lot of data is needed for models to perform correctly. Also, 1-hots are not able to portray the similarities between words, losing relevant information.

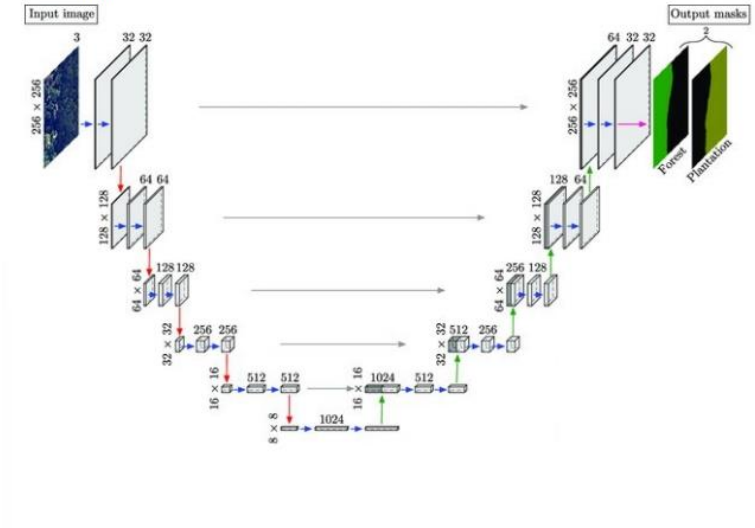
A solution can be word embedding, which is a technique used to map through a NN the word representations int a lower dimensional space, reducing the complexity and also being able to capture the closeness in meanings of words, so that similar words are also closer in the latent space.

What is the pain behind the input of variable sized images in CNNs? What is the relief?

CNNs are NNs that use a layered structure with the presence of Conv layers, which are layers that use a set of filters, or kernels, to extract relevant features. The last layers in the CNNs are however FC layers, and this is because these nets usually needs to perform classic operations of classification and regression. Because of this, they require a fixed size to be processed correctly. A possible solution can be inserting a GAP layers, that generates a single value for each feature map, and have it sized to the adequate value, or using 1x1 convolutions to make the NN a fully connected one.

QUESTION 3: CONVOLUTIONAL NEURAL NETWORKS

With reference to the deep neural network in the image below, answer the following questions.



Blue arrows refer to convolutions having a spatial extent 3x3, while the magenta one does not perform spatial averages

Input	(256, 256, 3)	0
Conv2D	(256, 256, 32)	(3 * 3 * 3 + 1) * 32
Conv2D	(256, 256, 32)	(3 * 3 * 32 + 1) * 32
MaxPooling2D	(128, 128, 32)	0
Conv2D	(128, 128, 64)	(3 * 3 * 32 + 1) * 64
Conv2D	(128, 128, 64)	(3 * 3 * 64 + 1) * 64
MaxPooling2D	(64, 64, 64)	0
Conv2D	(64, 64, 128)	(3 * 3 * 64 + 1) * 128
Conv2D	(64, 64, 128)	(3 * 3 * 128 + 1) * 128
MaxPooling2D	(32, 32, 128)	0
Conv2D	(32, 32, 256)	(3 * 3 * 128 + 1) * 256
Conv2D	(32, 32, 256)	(3 * 3 * 256 + 1) * 256
MaxPooling2D	(16, 16, 256)	0
Conv2D	(16, 16, 512)	(3 * 3 * 256 + 1) * 512
Conv2D	(16, 16, 512)	(3 * 3 * 512 + 1) * 512

MaxPooling2D	(8, 8, 512)	0
Conv2D	(8,8,1024)	(3 * 3 * 512 + 1) * 1024
Conv2D	(8,8,1024)	(3 * 3 * 1024 + 1) * 1024
Upsampling2D	(16, 16, 1024)	0

Specchio dei primi layer per il resto della rete

What is the task this network is expected to solve? What is the rationale behind this architecture and which loss function would you would use to train the network? Justify all your statements.

This net is built to solve image segmentation, which is the task of assigning a class value to each pixel in an image. It does this by extracting features and then building the feature maps through upsampling and skip connections. A possible loss function can be the crossentropy loss, which is suitable for classification problems.

How does a Neural Touring Machine work?

A NTM is a network that was built to be able to access an external memory: this memory is in the form of an external memory matrix that this network is able to access. It is a model that uses this memory to process data while also taking into account past information. Weights are used to identify where to write and read from memory.

How does the WRITE mechanism works in a Neural Touring Machine?

A NTM uses a set of write heads to access the memory cells. The write process influences all cells but to a different extent, thanks to an attention mechanism: weights are computed considering the present and past state of the network and used to assign relevance for each cell in the current operation.

How does the READ mechanism work in a Neural Touring Machine?

The READ mechanism works in a similar way: the net is able to learn a set of weights that are used to balance the relevance of each memory cell for the operation. This way, the input is generated using information that's organized to be more relevant.

What is sequence to sequence modeling? How does the attention mechanism is used in sequence to sequence modeling?

Seq2seq modeling is a type of NNs that uses models able to process data sequences. These models can capture the relationships between the elements of a sequence and extract features to make predictions and reconstructions. They use an encoder-decoder type of structure to extract information and process it. The encoder is able to represent a context vector in its hidden states and the decoder can use this representation as input. Attention is useful because it allows the decoder to focus on specific sections of the input, using a set of weights that give different relevance to different parts of it to make more accurate outcomes. These weights are generated by considering the hidden states of the encoder and the state of the decoder, and using these values to output a weight distribution over the vector.

03/09/2020

Is data representation, i.e., features, learned via deep learning always better than hand-crafted features? Why?

Usually, features extracted using DL are able to represent more complex characteristics of the dataset, thus giving a more meaningful and robust representation. However, it's not always guaranteed that they will be better. Hand-crafted features are controlled by the designer and may be more understandable compared to the computed ones, also they could be more fitting to the solution than the DL ones. DL requires a lot of data to perform, while hand crafted features do not have this necessity.

How data representation is learned via convolutional neural networks?

CNN learn data representation using a particular structure that's designed to work with grid-shaped elements like images. They use a set of convolutional operation, which consist of sliding some filters, or kernels, over the image, each of which performs a linear operation (dot product) over the input and generates what's called a feature map. Each kernel is used to spot a different feature in the image, to different levels of abstraction according to the layer's depth, and combining them we are able to extract meaningful representations of data.

How data representation is leaned via recurrent neural network?

RNNs are able to learn data representation by using a particular structure: in fact, differently from FFNN, these nets are able to keep memory of the past values and use them to generate new outputs. The output is in this way a function of the input and of the past hidden states. They use a standard net, which is FF, and a context net, which is used as memory and has recurrent connections. The context net has a layer for each timestep, and each layer represents the state of the net in a particular instant. Backpropagation is performed in a way that's called BPTT, because in the context net weights are shared across each timestep. This can make them suffer from vanishing gradient more easily.

How data representation is learned via deep autoencoders?

Deep autoencoders learn data representations thanks to their particular structure, in an unsupervised manner. They are made of 2 sections, an encoder and a decoder: the encoder takes an input and using multiple FC or Conv layers tries to extract the most relevant features, so that it's able to map it in a smaller latent space, and the decoder, starting from it, is able to reconstruct it and bring it back to the original space through FC or TransposeConv layers. DA have a rather unstable training process, and their performance is measured using a loss function like MSE: ||s-D(E(s))||_2.

What is overfitting? What it is due to?

Overfitting is a phenomenon that can manifest itself in NNs. It consists in the NN being too adapted to the training data, so that its performance on it is optimal, while when fed unknown data it becomes lacking, meaning that it has not the proper generalization capabilities. This can be due to a variety of reasons, such as the time spent for training (it should not be too long), the number of parameters in the NN, the dimension of the training set, the weight initialization.

What is early stopping and why does it help with overfitting?

Early stopping can help prevent overfitting. It consists of choosing some hyperparameters like a metric to monitor and the number of epochs after which it has not improved (patience), and after which the training needs to be stopped. It is usually performed with cross-validation, because the validation error is a useful metric to monitor when the NN is starting to underperform on unseen data. So even if the desired number of epochs for training has not been reached yet, it is stopped to avoid losing generalization capabilities.

What is weight decay and why does it help with overfitting?

Weight decay is a technique used to avoid overfitting: it consists of putting a penalty term in the loss function, so that the norms of the weights are kept under control during training and they do not assume values trat are too high. This term is usually generated after making an a-priori assumption over the weight distribution, and its importance is balanced by a gamma hyperparameter which has to be tuned for optimal performance. This is useful because having too large weights tends to make the NN adapt too much to the training values, while smaller weights reduce this issue.

What is dropout and why does it help with overfitting?

Dropout is a technique that's useful to prevent overfitting. It's actuated by deactivating some neurons during training according to a probability hyperparameter: tis way, the net is forced to learn redundant representations of the dataset, and this makes it more robust to training noise. It's like having multiple subnets which are averaged together to obtain the final model.

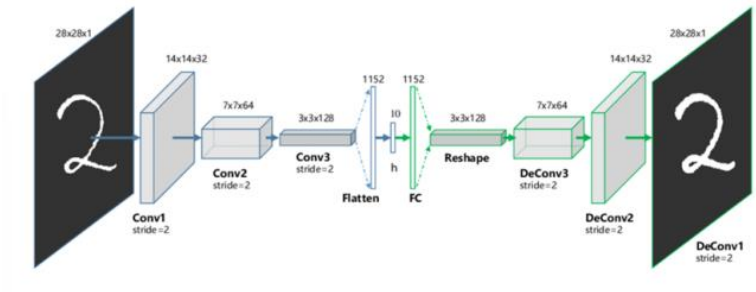
Detail the building blocks in each layer of the above network. Please indicate

- what kind of block is

- what is its size

- what is the overall number of parameters, together with a short description on how you compute them, e.g., 3x5x5=45 (not just 45!).

Consider convolutions having a spatial 3x3 extent. You can use the calculator, but we are more interested in the formulas than on numbers!



Input	(28, 28, 1)	0
Conv2D	(14, 14, 32)	(3 * 3 * 1 + 1) * 32
Conv2D	(7, 7, 64)	(3 * 3 * 32 + 1) * 64
Conv2D	(3, 3, 128)	(3 * 3 * 64 + 1) * 128
Flatten	(1152,)	0
Dense	(10,)	1152*10+10
Dense	(1152,)	1152*10+1152
Reshape	(3, 3, 128)	0
DeConv	(7, 7, 64)	(3*3*128+1)*64
DeConv	(14, 14, 32)	(3 * 3 * 64 + 1) * 32
DeConv	(28, 28, 1)	(3 * 3 * 32 + 1) * 1

This network is called denoising autoencoder and it has the goal of cleaning or restoring a damaged image. How would you train it? What kind of loss function would you use?

The training can be performed by feeding this net images of the desired training set but with some noise added, so that it can learn a robust latent representation and use it to reconstruct the output. The performance of the model can be measured by using a function that takes into account the distance between the original input and the output of the autoencoder. One example can be the MSE calculated like: ||s-D(E(s))||_2.

Do I really need a recurrent neural network to process a stream of input data? Why can't I use a standard feed forward architecture?

A FF architecture is not able to process data sequences. That's because they are not able to keep a memory of past inputs, and this is necessary for sequential data since it can highlight meaningful relationships. The FF architecture only considers one example at a time, so others are not taken into consideration, while RNN, thanks to their context network, can use that information to compute new outputs.

Classic recurrent neural networks suffer the vanishing gradient problem. What is it and what is it due to? Does it affects only recurrent networks?

Vanishing gradient is a phenomenon that can affect RNNs. They are particularly prone to this issue because of their deep and complex structure, having a layer for each timestep. It is mostly due to the use of some activation functions like Sigmoid and Tanh, which have their derivatives between 0 and 1: when they are multiplied backwards during BPTT, their small values make the gradient of the earlier layers shrink, since the weights are shared between timesteps. This often leads to a slow convergence and an inefficient update of weights. However, also NNs are affected by it, especially if they use those same activations hìand have an incorrect weight initialization and deep structures.

What is ReLU and why it helps with vanishing gradient?

ReLU is a kind of activation function commonly used in NNs. It's often found when dealing with regression problems as the output function. Its main perk is that it has a derivative that can only take values of 0 and 1: in this way, the vanishing gradient is not an issue, because the values cannot shrink. However, if the inputs of the functions are in saturation zones, their derivative can become fixed to 0, and this causes the

weights to not be updated anymore, so that the neurons become dead, since they do not learn from training anymore.

What is Xavier initialization and why it helps with vanishing gradient?

Xavier initialization is a technique for weight initialization. Performing a correct weight initialization is necessary because they are linked to the performance of the NN, and they may lead to overfitting, underfitting or other issues. Vanishing gradient may be caused by selecting weights that are too small, even if smaller values are usually preferred to larger ones, but in deep NNs they tend to shrink too much. Xavier initialization selects the weights in a way that they're sampled from a gaussian distribution centered in 0. Its variance is equal to 1/n_i where n_i is the number of inputs of the layer. This helps because the weights values are kept under control and they're not reduced too much.

What is an LSTM and why it helps with vanishing gradient?

LSTM is a particular kind of RNN that is able to keep memory of past inputs thanks to its particular structure: it uses a set of gates to manipulate the long term and short term memory that it uses in its hidden layers for computations. It eliminates vanishing and exploding gradient issues because of the CEC: this is a mechanism that is able to keep the gradient of the error to 1. It also uses linear activations to preserve the values and not reduce them too much. The NN is still able to learn thanks to its weighted gates.

26/01/2023

Vanishing and exploding gradients are issues of Neural Network Training. For each of the following statements, you have to state if it is True or False.

1. Shortcut (or skip) connections are used to face the vanishing and exploding gradients.

True. Shortcut connections, also known as residual connections, are used to address the problem of vanishing and exploding gradients in deep neural networks. These connections allow gradients to bypass one or more layers in the network, which can help to preserve the gradient information and make it easier for the model to learn. This technique is known as Residual learning or ResNet which is used to train very deep neural networks successfully.

2. The vanishing gradient is present just in recurrent neural networks (RNN).

False. Vanishing gradients can occur in both feedforward neural networks (FFNN) and recurrent neural networks (RNN) especially when the network is deep. In RNNs, the problem is more pronounced because the gradients must pass through many time steps, which can cause the gradients to become very small.

3. Batch normalization has been introduced to solve the vanishing gradient issue.

False. Batch normalization is a technique that is used to reduce the internal covariate shift and to

stabilize the training of deep neural networks. It does this by normalizing the activations of each layer to have zero mean and unit variance. Batch normalization does not directly address the issue of vanishing gradients, but it can make the network more robust to the initialization of the parameters, which may allow the gradients to propagate more easily through the layers.

4. Rectified Linear Unit (i.e., ReLUs) are useful in mitigating exploding gradient but not vanishing gradient.

False. ReLU (Rectified Linear Unit) activation function, which outputs the input if it is positive, and 0 otherwise, can solve both issues. Indeed, the derivative of ReLU is 1, besides when there is a dead neuron issue, this helps in both cases.

5. Rectified Linear Unit (i.e., ReLUs) are useful in mitigating vanishing gradient but not exploding gradient.

False. ReLU (Rectified Linear Unit) activation function, which outputs the input if it is positive, and 0 otherwise, can solve both issues. Indeed, the derivative of ReLU is 1, besides when there is a dead neuron issue, this helps in both cases.

6. The exploding gradient is solved in long short-term memories (LSTM).

True. Long Short-Term Memory (LSTM) networks are a type of recurrent neural network (RNN) that are designed to overcome the vanishing gradient problem by introducing a memory cell, gates, and a cell state. These components allow the LSTM to selectively store and access information, and to prevent gradients from exploding or vanishing during backpropagation.

You have to train a feedforward neural network with a limited amount of data and you want to use as much as possible of these. Describe how you could train the model to prevent overfitting and motivate your choices in light of the previous statement.

The core of this question is on the reduced amount of data so you should use techniques to limit overfitting which do not require the hold out of data as much as possible. Proper methods are dropout and weight decay, while early stopping requires you to waste data and the exercise was exactly about avoiding this!

For hyperparameter tuning, e.g., for the lambda in weight decay, k-fold/holdout/loo can be used, but once the proper parameter has been selected all data should be put back together and the model should be retrained. Also, Data Augmentation and Transfer Learning could be used to face the problem of limited data.

Before the invention of Transformers, seq2seq models with attention where the state of art in neural language translation.

a) Describe the seq2seq model without attention

b) How is attention added to seq2seq models? Why it was added?

c) What is word2vec? How it is related to seq2seq models?

a) A seq2seq model without attention is a type of neural network architecture that is used for tasks such as language translation, text summarization, and image captioning. The basic architecture consists of two main parts: an encoder, which takes in a sequence of input data and produces a fixed-length context vector, and a decoder, which takes the context vector and produces a sequence of output data. The encoder and decoder are typically implemented using recurrent neural networks (RNNs) such as LSTMs or GRUs. The encoder's job is to encode the input sequence into a single vector, called a context vector, which contains information about the entire input sequence. The decoder then uses this context vector to generate the output sequence. Training is performed by providing the proper sequence to the decoder (teacher forcing) while at inference time an autoregressive approach is used providing as input the word just predicted by the decoder.

b) Attention is added to seq2seq models to improve the ability of the model to focus on specific parts of the input sequence when generating the output sequence. Attention mechanisms work by allowing the decoder to "attend" to different parts of the input sequence at different time steps, rather than using a fixed context vector. Attention mechanisms are implemented by introducing a new component, called an attention mechanism, that learns to assign weights to different parts of the input sequence at each time step. The attention mechanism is typically implemented using a feedforward neural network, which takes as input the current decoder state and the encoder states and produces a set of attention weights. These attention weights are then used to weigh the encoder states when generating the next output. Attention was added to seq2seq models to improve the performance on tasks that require the model to focus on specific parts of the input when generating the output, such as language translation and text summarization.

c) Word2vec is a technique for learning vector representations of words, also known as word embeddings. It is a neural network-based technique that learns to predict the context of a given word based on its surrounding words. Word2vec is related to seq2seq models in the sense that it is often used as a pre-training step for the encoder in a seq2seq model. The embeddings learned by word2vec can be used to initialize the embedding layer of the encoder, which can improve the performance of the seq2seq model on tasks such as language translation and text summarization.

Assume you have to train a CNN that

- takes as input a grayscale picture of a binary number as the one below, and

- predicts the value of the encoded number

Please mark all the sentences that are correct.

Scegli una o più alternative:

a. Normalizing the data before training can be beneficial. This is true, like other inference problems on images.

b. I can perform data augmentation by zooming, resizing and resampling the input image at training time. True but no explanation

c. It is convenient to train the network minimizing categorical cross entropy, as long as the number to be predicted are positive.

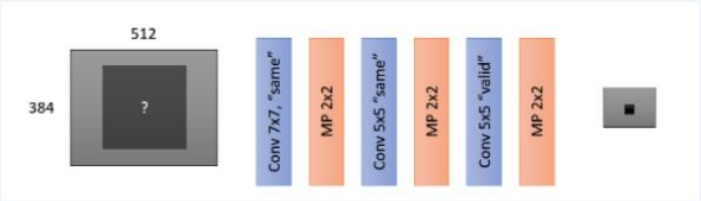
d. I can perform data augmentation by translating the image and by minor perspective changes. Sure, these should not change the associated label.

e. I can perform data augmentation by vertical and horizontal flips and by adding moderate amount of blur and noise.

f. I can select a portion of data as validation set and use this to assess stopping criteria. Sure, this is a good practice to have a validation set to assess stopping criteria.

Receptive Field

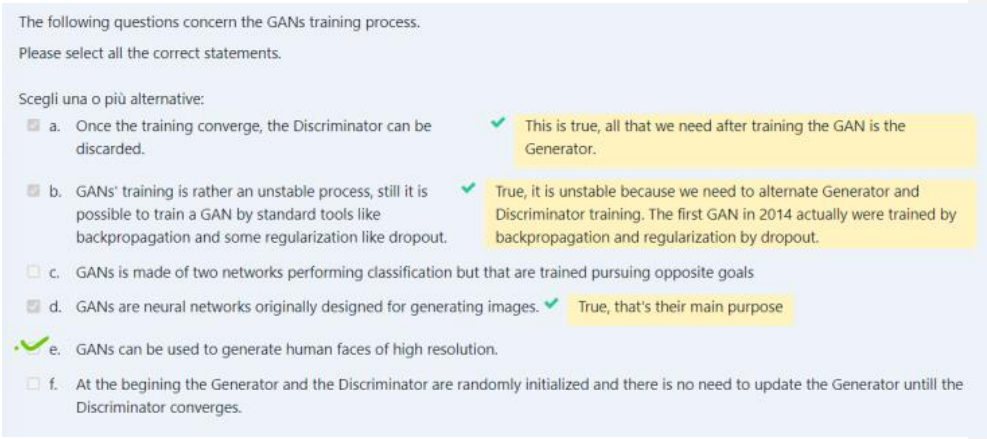
What is the receptive field at the central location of the activation map after the followign layers?



Scegli un'alternativa:

- a. 42x42
- b. 24x24
- c. 28x42
- d. it is not possible to say since the output size are not defined
- e. 38x38 ✔

GAN Training



The following questions concern the GANs training process.

Please select all the correct statements.

Scegli una o più alternative:

a. Once the training converge, the Discriminator can be discarded. ✔ This is true, all that we need after training the GAN is the Generator.

b. GANs' training is rather an unstable process, still it is possible to train a GAN by standard tools like backpropagation and some regularization like dropout. ✔ True, it is unstable because we need to alternate Generator and Discriminator training. The first GAN in 2014 actually were trained by backpropagation and regularization by dropout.

c. GANs is made of two networks performing classification but that are trained pursuing opposite goals

d. GANs are neural networks originally designed for generating images. ✔ True, that's their main purpose

e. GANs can be used to generate human faces of high resolution.

f. At the begining the Generator and the Discriminator are randomly initialized and there is no need to update the Generator untill the Discriminator converges.

Le risposte corrette sono:

GANs are neural networks originally designed for generating images.

GANs' training is rather an unstable process, still it is possible to train a GAN by standard tools like backpropagation and some regularization like dropout.,

Once the training converge, the Discriminator can be discarded.,

GANs can be used to generate human faces of high resolution

Architecture

Consider the following code

```
import tensorflow as tf
tfkl = tf.keras.layers
tfk = tf.keras
def get_model_1(input_shape, num_classes):
    input_layer = tfkl.Input(shape=input_shape, name='input_layer')
    cd1 = tfkl.Conv2D(128, kernel_size=3, padding='same', activation='relu', name='conv_d1')(input_layer)
    d1 = tfkl.MaxPooling2D(name='mp_d1')(cd1)
    cd2 = tfkl.Conv2D(256, kernel_size=3, padding='same', activation='relu', name='conv_d2')(d1)
    d2 = tfkl.MaxPooling2D(name='mp_d2')(cd2)
    cd3 = tfkl.Conv2D(512, kernel_size=3, padding='same', activation='relu', name='conv_d3')(d2)
    u1 = tfkl.UpSampling2D(name='up_u1')(cd3)
    u1 = tfkl.Concatenate(name='concat1')([u1,cd2])
    u1 = tfkl.Conv2D(256, kernel_size=3, padding='same', activation='relu', name='conv_u1')(u1)
    u2 = tfkl.UpSampling2D(name='up_u2')(u1)
    u2 = tfkl.Concatenate(name='concat2')([u2,cd1])
    u2 = tfkl.Conv2D(128, kernel_size=3, padding='same', activation='relu', name='conv_u2')(u2)
    output_layer = tfkl.Conv2D(num_classes, kernel_size=3, padding='same', activation="softmax", name='output_layer')(u2)

    model = tf.keras.Model(inputs=input_layer, outputs=output_layer)
    model.compile(
        loss = tfk.losses.SparseCategoricalCrossentropy(),
        optimizer = tfk.optimizers.Adam(),
        metrics = [
            tfk.metrics.Accuracy(),
            tfk.metrics.MeanIoU(num_classes=num_classes)
            ]
        )
    return model
```

```
input_shape = (128,256,3)
num_classes = 3
model = get_model_1(input_shape=input_shape, num_classes=num_classes)
s=model.summary()


The last command returns the following summary, which we provide for reference
_________________________________________________________________________________________________
 Layer (type)                Output Shape              Param #     Connected to
=================================================================================================
 input_layer (InputLayer)
 conv_d1 (Conv2D)
 mp_d1 (MaxPooling2D)
 conv_d2 (Conv2D)
 mp_d2 (MaxPooling2D)
 conv_d3 (Conv2D)
 up_u1 (UpSampling2D)
 concat1 (Concatenate)
 conv_u1 (Conv2D)
 up_u2 (UpSampling2D)
 concat2 (Concatenate)
 conv_u2 (Conv2D)
 output_layer (Conv2D)
=================================================================================================
Total params:
Trainable params:
Non-trainable params:
_________________________________________________________________________________________________
```

Solution:

```
 Layer (type)                Output Shape              Param #     Connected to
=================================================================================================
 input_layer (InputLayer)    [(None, 128, 256, 3)]     0           []
 conv_d1 (Conv2D)            (None, 128, 256, 128)     3584        ['input_layer[0][0]']
 mp_d1 (MaxPooling2D)        (None, 64, 128, 128)      0           ['conv_d1[0][0]']
 conv_d2 (Conv2D)            (None, 64, 128, 256)      295168      ['mp_d1[0][0]']
 mp_d2 (MaxPooling2D)        (None, 32, 64, 256)       0           ['conv_d2[0][0]']
 conv_d3 (Conv2D)            (None, 32, 64, 512)       1180160     ['mp_d2[0][0]']
 up_u1 (UpSampling2D)        (None, 64, 128, 512)      0           ['conv_d3[0][0]']
 concat1 (Concatenate)       (None, 64, 128, 768)      0           ['up_u1[0][0]', 'conv_d2[0][0]']
 conv_u1 (Conv2D)            (None, 64, 128, 256)      1769728     ['concat1[0][0]']
 up_u2 (UpSampling2D)        (None, 128, 256, 256)     0           ['conv_u1[0][0]']
 concat2 (Concatenate)       (None, 128, 256, 384)     0           ['up_u2[0][0]', 'conv_d1[0][0]']
 conv_u2 (Conv2D)            (None, 128, 256, 128)     442496      ['concat2[0][0]']
 output_layer (Conv2D)       (None, 128, 256, 3)       3459        ['conv_u2[0][0]']
=================================================================================================
Total params: 3,694,595
Trainable params: 3,694,595
Non-trainable params: 0
```

Mark all the tasks that the above network can be trained for.

Please, refrain from marking answers that are "technically possible in principle in Python", and stick to those tasks that a Neural Network expert (as you are expected to be) would use this network for.

Scegli una o più alternative:

a. Weather forecasting

b. Given an input grayscale image, bring this back to colors

c. Counting the number of cars passing on a street.

d. Determining which pixels in an image belongs to road, sky and anything that does not belong to these classes

e. Determining which pixels in an image of a checkerboard are white, which are black and which are covered any piece.

f. Given an histological image, determining pixels covered by cells, tubules and interstitial area between cells.

g. Determining which pixels in an image belongs to car and which belongs to other.

It's a 3 class segmentation problem. The correct answers are:

Determining which pixels in an image of a chekerboard are white, which are black and which are covered any piece.,

Determining which pixels in an image belongs to road, sky and anything that does not belong to these clasess.,

Given an histological image, determining pixels coverd by cells, tubules and interstitial area between cells